

Contents

1	Part A: Statistical Inference	3
1.1	Problem 1	3
1.1.1	(a)	3
1.1.2	(b)	4
1.1.3	(c)	4
1.1.4	(d)	6
1.1.5	(e)	8
1.1.6	(f)	9
1.1.7	(g)	11
1.1.8	(h)	13
1.2	Problem 2	17
2	Part B: Data Analysis	30
2.1	Data pre-processing and experimental setup	30
2.1.1	Data Analysis	30
2.2	Poisson regression	32
2.2.1	Stepwise model selection	32
2.2.2	L_1 -Lasso Regularization	35
2.2.3	Discussion	36
2.3	Logistic Regression	37
2.3.1	Stepwise Model Selection	37
2.3.2	L_1 -Lasso Regularization	38
2.3.3	Discussion	39

2.4 Discussion — Differences Between L_1 -regularization and Stepwise Model Selection	41
A Appendix	43
A.1 Statement on the Useage of Gen. AI	43
A.2 Additional Tables	44
A.2.1 Proof of the dependency of f_1 and f_2	45
A.3 Additional Plots and Tables	48

Chapter 1

Part A: Statistical Inference

1.1 Problem 1

1.1.1 (a)

Express $P(X_i = 1)$ as a function of a and π .

We have

$$\begin{aligned} P(X_i = 1) &= P(X_i = 1, \text{Group} = A) + P(X_i = 1, \text{Group} = B) \\ &= P(X_i = 1 \mid \text{Group} = A) \cdot P(\text{Group} = A) + P(X_i = 1 \mid \text{Group} = B) \cdot P(\text{Group} = B) \\ &= a \cdot P(\text{Group} = A) + (1 - a) \cdot P(\text{Group} = B) \\ &= a \cdot \pi + (1 - a) \cdot (1 - \pi) . \end{aligned}$$

Similarly, we have

$$\begin{aligned} P(X_i = 0) &= P(X_i = 0, \text{Group} = A) + P(X_i = 0, \text{Group} = B) \\ &= P(X_i = 0 \mid \text{Group} = A) \cdot P(\text{Group} = A) + P(X_i = 0 \mid \text{Group} = B) \cdot P(\text{Group} = B) \\ &= (1 - a) \cdot P(\text{Group} = A) + a \cdot P(\text{Group} = B) \\ &= (1 - a) \cdot \pi + a \cdot (1 - \pi) . \end{aligned}$$

(Optional) Because the random variable X_i can only have 2 potential values, we can now easily check whether

our expressions are correct as $P(X_i = 1) + P(X_i = 0) = 1$. We have

$$\begin{aligned} P(X_i = 1) + P(X_i = 0) &= a \cdot \pi + (1 - a) \cdot (1 - \pi) + (1 - a) \cdot \pi + a \cdot (1 - \pi) \\ &= a \cdot \pi + (1 - \pi) - a + \pi \cdot a + \pi - \pi \cdot a + a - \pi \cdot a \\ &= a \cdot \pi + 1 - \pi + \pi - \pi \cdot a \\ &= 1 . \end{aligned}$$

1.1.2 (b)

Give expressions for the joint probability of the observed data $X_1, X_2, \dots, X_n$, and the corresponding likelihood function and loglikelihood function.

We aim to express the computation for

$$P(X_1, X_2, \dots, X_n) .$$

Because we know that the n samples are drawn with the i.i.d. assumption, it holds that

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i) .$$

Hence,

$$\begin{aligned} P(X_1, X_2, \dots, X_n) &= \prod_{i=1}^n P(X_i) \\ &= \prod_{i=1}^n \begin{cases} P(X_i = 1) & \text{if } X_i = 1 \\ P(X_i = 0) & \text{if } X_i = 0 \end{cases} \\ &= P(X_i = 1)^m \cdot P(X_i = 0)^{n-m} \\ &= (a \cdot \pi + (1 - a) \cdot (1 - \pi))^m \cdot ((1 - a) \cdot \pi + a \cdot (1 - \pi))^{n-m} \end{aligned} \quad (1.1)$$

is the likelihood function of the observed samples.

Let us now derive the expression for the log likelihood of the data:

$$\begin{aligned} \log P(X_1, X_2, \dots, X_n) &= \log \left[(a \cdot \pi + (1 - a) \cdot (1 - \pi))^m \cdot ((1 - a) \cdot \pi + a \cdot (1 - \pi))^{n-m} \right] \\ &= m \cdot \log [a \cdot \pi + (1 - a) \cdot (1 - \pi)] + (n - m) \cdot \log [(1 - a) \cdot \pi + a \cdot (1 - \pi)] \end{aligned} \quad (1.2)$$

1.1.3 (c)

Derive an expression for the maximum likelihood estimator π_{ML} of π .

We now aim to estimate π . To this end, we denoted the estimated value for π as π_{ML} in Equation 1.2 and

derive the log likelihood w.r.t. π_{ML} . We have

$$\begin{aligned} & \frac{\partial}{\partial \pi_{ML}} \log P(X_1, X_2, \dots, X_n) \\ &= \frac{\partial}{\partial \pi_{ML}} m \cdot \log [a \cdot \pi_{ML} + (1-a) \cdot (1-\pi_{ML})] + (n-m) \cdot \log [(1-a) \cdot \pi_{ML} + a \cdot (1-\pi_{ML})] \\ &= m \cdot \frac{1}{a \cdot \pi_{ML} + (1-a) \cdot (1-\pi_{ML})} \cdot (a - (1-a)) + (n-m) \cdot \frac{1}{(1-a) \cdot \pi_{ML} + a \cdot (1-\pi_{ML})} \cdot ((1-a) - a). \end{aligned}$$

We now find the critical points by setting the gradient to zero:

$$\begin{aligned} & \frac{\partial}{\partial \pi_{ML}} \log P(X_1, X_2, \dots, X_n) \stackrel{!}{=} 0 \\ m \cdot \frac{1}{a \cdot \pi_{ML} + (1-a) \cdot (1-\pi_{ML})} \cdot (a - (1-a)) + (n-m) \cdot \frac{1}{(1-a) \cdot \pi_{ML} + a \cdot (1-\pi_{ML})} \cdot ((1-a) - a) &= 0 \end{aligned} \quad (1.3)$$

where Equation 1.3 holds whenever $(1-a) = a$ as both additive terms are zero. It is easy to see that $a = 0.5$ is the only real solution for this equation. However, as we are interested in the maximum likelihood estimator of π_{ML} , setting $a = 0.5$ prohibits us from finding the maximum value. This is because if $a = 0.5$, we have $P(X_i = 1) = 0.5 = P(X_i = 0)$, and therefore the likelihood function is constant, regardless of the value of π . Therefore, we need to consider the case where $a \neq 0.5$.

We now solve Equation 1.3 for π_{ML} while assuming that $q \neq 0.5$. We have:

$$\begin{aligned} m \cdot \frac{1}{a \cdot \pi_{ML} + (1-a) \cdot (1-\pi_{ML})} \cdot (a - (1-a)) &= -(n-m) \cdot \frac{1}{(1-a) \cdot \pi_{ML} + a \cdot (1-\pi_{ML})} \cdot ((1-a) - a) \\ m \cdot \frac{1}{a \cdot \pi_{ML} + (1-a) \cdot (1-\pi_{ML})} \cdot (a - (1-a)) &= (n-m) \cdot \frac{1}{(1-a) \cdot \pi_{ML} + a \cdot (1-\pi_{ML})} \cdot (a - (1-a)) \\ m \cdot \frac{1}{a \cdot \pi_{ML} + (1-a) \cdot (1-\pi_{ML})} &= (n-m) \cdot \frac{1}{(1-a) \cdot \pi_{ML} + a \cdot (1-\pi_{ML})} \\ m \cdot \frac{1}{(2a-1) \cdot \pi_{ML} + 1-a} &= (n-m) \cdot \frac{1}{(1-2a) \cdot \pi_{ML} + a} \\ m \cdot \frac{1}{(2a-1) \cdot \pi_{ML} + 1-a} &= (n-m) \cdot \frac{1}{a - (2a-1) \cdot \pi_{ML}} \\ m \cdot (a - (2a-1) \cdot \pi_{ML}) &= (n-m) \cdot ((2a-1) \cdot \pi_{ML} + 1-a) \\ am - (2a-1) \cdot m \cdot \pi_{ML} &= (n-m) \cdot (2a-1) \cdot \pi_{ML} + (n-m) \cdot (1-a) \\ am - (n-m) \cdot (1-a) &= (n-m) \cdot (2a-1) \cdot \pi_{ML} + (2a-1) \cdot m \cdot \pi_{ML} \\ am - (n-m) \cdot (1-a) &= \pi_{ML} \cdot (2a-1)((n-m) + m) \\ am - (n-m) + a \cdot n - am &= \pi_{ML} \cdot (2a-1)(n) \\ n \cdot (a-1) + m &= \pi_{ML} \cdot (2a-1)(n) \\ \frac{n \cdot (a-1) + m}{(2a-1)(n)} &= \pi_{ML} \\ \frac{n \cdot (a-1) + m}{n} \cdot \frac{1}{2a-1} &= \pi_{ML} \\ \left(a-1 + \frac{m}{n}\right) \cdot \frac{1}{2a-1} &= \pi_{ML} \end{aligned} \quad (1.4)$$

We verify that this is indeed a maximum by plugging the result from Equation 1.4 into the second derivative. For ease of notation, we will use $b = (1 - a)$. We have

$$\begin{aligned}
 & \frac{\partial^2}{\partial \pi_{ML}^2} \log P(X_1, X_2, \dots, X_n) \\
 &= \frac{\partial}{\partial \pi_{ML}} m \cdot (a - (1 - a)) \cdot \frac{1}{a \cdot \pi_{ML} + (1 - a) \cdot (1 - \pi_{ML})} + (n - m) \cdot ((1 - a) - a) \cdot \frac{1}{(1 - a) \cdot \pi_{ML} + a \cdot (1 - \pi_{ML})} \\
 &= -m \cdot (a - (1 - a)) \cdot \frac{1}{(a \cdot \pi_{ML} + (1 - a) \cdot (1 - \pi_{ML}))^2} \cdot (a - (1 - a)) \\
 &\quad - (n - m) \cdot ((1 - a) - a) \cdot \frac{1}{((1 - a) \cdot \pi_{ML} + a \cdot (1 - \pi_{ML}))^2} \cdot ((1 - a) - a) \\
 &= -m \cdot (a - (1 - a)) \cdot \frac{1}{(a \cdot \pi_{ML} + (1 - a) \cdot (1 - \pi_{ML}))^2} \cdot (a - (1 - a)) \\
 &\quad - (n - m) \cdot (a - (1 - a)) \cdot \frac{1}{((1 - a) \cdot \pi_{ML} + a \cdot (1 - \pi_{ML}))^2} \cdot (a - (1 - a)) \\
 &= -m \cdot (a - (1 - a))^2 \cdot \frac{1}{(a \cdot \pi_{ML} + (1 - a) \cdot (1 - \pi_{ML}))^2} - (n - m) \cdot (a - (1 - a))^2 \cdot \frac{1}{((1 - a) \cdot \pi_{ML} + a \cdot (1 - \pi_{ML}))^2} \\
 &= -m \cdot (a - b)^2 \cdot \frac{1}{(a \cdot \pi_{ML} + b \cdot (1 - \pi_{ML}))^2} - (n - m) \cdot (a - b)^2 \cdot \frac{1}{(b \cdot \pi_{ML} + a \cdot (1 - \pi_{ML}))^2} \\
 &\leq -m \cdot \frac{1}{(a \cdot \pi_{ML} + b \cdot (1 - \pi_{ML}))^2} - (n - m) \cdot \frac{1}{(b \cdot \pi_{ML} + a \cdot (1 - \pi_{ML}))^2} \\
 &= -m \cdot \frac{1}{(a \cdot \pi_{ML} + (1 - a) \cdot (1 - \pi_{ML}))^2} - (n - m) \cdot \frac{1}{((1 - a) \cdot \pi_{ML} + a \cdot (1 - \pi_{ML}))^2} \\
 &= -m \cdot \alpha - (n - m) \cdot \beta \\
 &\leq -m - (n - m) \\
 &\leq 0
 \end{aligned}$$

with $\alpha > 0$ and $\beta > 0$ from the squaring of the denominator. Hence, we know that the second derivative is negative whenever $n > 0$. From this, it follows that our estimator π_{ML} is indeed a maximum.

1.1.4 (d)

Can you think of an easier way to arrive at the same estimator as the maximum likelihood estimator?

We observe that X_i follows a Bernoulli distribution. We now first show that this holds.

Recall that the definition of a Bernoulli distribution is given by

$$\begin{aligned}
 P(X = 1) &= p \\
 P(X = 0) &= 1 - p.
 \end{aligned}$$

Therefore, we now aim to show that $P(X_i)$ follows a Bernoulli distribution. From (a), we have that We have

$$P(X_i = 1) = a \cdot \pi + (1 - a) \cdot (1 - \pi)$$

and

$$P(X_i = 0) = (1 - a) \cdot \pi + a \cdot (1 - \pi) .$$

We multiply out the expression to get

$$\begin{aligned} P(X_i = 1) &= a \cdot \pi + 1 - \pi - a \cdot (1 - \pi) \\ &= a \cdot \pi + 1 - \pi - a + a \cdot \pi \\ &= 2a \cdot \pi + 1 - \pi - a \\ &= 1 + 2a \cdot \pi - \pi - a \end{aligned} \tag{1.5}$$

$$\begin{aligned} P(X_i = 0) &= (1 - a) \cdot \pi + a \cdot (1 - \pi) \\ &= \pi - a \cdot \pi + a - a \cdot \pi \\ &= -2a \cdot \pi + \pi + a . \end{aligned} \tag{1.6}$$

Therefore, with $p = 1 + 2a \cdot \pi - \pi - a$ (from Equation 1.5), we have

$$\begin{aligned} P(X = 0) &= 1 - p \\ &= 1 - P(X = 1) \\ &= 1 - (1 + 2a \cdot \pi - \pi - a) \\ &= 1 - 1 - 2a \cdot \pi + \pi + a \\ &= -2a \cdot \pi + \pi + a \end{aligned} \tag{1.7}$$

where Equation 1.7 matches the expression in Equation 1.6. Therefore, we have shown that X_i follows a Bernoulli distribution with parameter $p = 1 + 2a \cdot \pi - \pi - a$.

Following from this result, because the random variable m is the sum of n i.i.d. Bernoulli random variables, we know that m follows a Binomial distribution with parameters n and p . Hence, the likelihood of observing m ones in n samples is given by the probability density function of the Binomial distribution:

$$\Pr(X = m) = \binom{n}{m} p^m (1 - p)^{n-m} . \tag{1.8}$$

While Equation 1.8 differs from the likelihood function we derived in Equation 1.1, we can see that the $\binom{n}{m}$ term cancels out when deriving the log likelihood; First, the log likelihood transforms the multiplication into a sum with a log term, and then when deriving w.r.t. π_{ML} , the $\log \binom{n}{m}$ becomes zero and therefore disappears. Hence, we end up with the same expression for the log likelihood.

As the Binomial distribution is well-known, we know that the maximum likelihood estimator of p of the Binomial distribution is given by $p_{ML} = \frac{m}{n}$, which is the proportion of successes, i.e., the number of $X_i = 1$ in n samples.¹

Finally, to derive the maximum likelihood estimator of π_{ML} , we equate p_{ML} and $P(X_i = 1)$ (as $P(X_i =$

¹See https://en.wikipedia.org/wiki/Binomial_distribution#Estimation_of_parameters.

1) = p), and then solve for π_{ML} :

$$\begin{aligned}
 P(X_i = 1) &= p_{ML} \\
 1 + 2a \cdot \pi_{ML} - \pi_{ML} - a &= \frac{m}{n} \\
 2a \cdot \pi_{ML} - \pi_{ML} &= \frac{m}{n} + a - 1 \\
 \pi_{ML} \cdot (2a - 1) &= \frac{m}{n} + a - 1 \\
 \pi_{ML} &= \frac{\frac{m}{n} + a - 1}{(2a - 1)} .
 \end{aligned} \tag{1.9}$$

We clearly observe that Equation 1.9 matches Equation 1.4, hence, we found the same maximum likelihood estimator of π_{ML} as in (c).

1.1.5 (e)

Show that π_{ML} is an unbiased estimator of π .

We aim to show that $\mathbb{E}[\pi_{ML}] = \pi$. To this end, we will make use of the knowledge introduced in (d), namely that X_i follows a Bernoulli distribution.

We have

$$\begin{aligned}
 \mathbb{E}[\pi_{ML}] &= \mathbb{E}\left[\frac{\frac{m}{n} + a - 1}{(2a - 1)}\right] \\
 &= \mathbb{E}\left[\frac{\frac{\sum_{i=1}^n X_i}{n} + a - 1}{(2a - 1)}\right] \\
 &= \frac{\sum_{i=1}^n \mathbb{E}[X_i] + a \cdot n - 1 \cdot n}{(2a - 1) \cdot n} .
 \end{aligned} \tag{1.10}$$

We now compute the expectation of X_i :

$$\begin{aligned}
 \mathbb{E}[X_i] &= P(X_i = 1) \cdot 1 + P(X_i = 0) \cdot 0 \\
 &= P(X_i = 1) \\
 &= a \cdot \pi + (1 - a) \cdot (1 - \pi) \\
 &= (1 - a) + (2a - 1) \cdot \pi .
 \end{aligned} \tag{1.11}$$

Finally, we can substitute Equation 1.11 into Equation 1.10 to get

$$\begin{aligned}
 \mathbb{E}[\pi_{ML}] &= \frac{\sum_{i=1}^n \mathbb{E}[X_i] + a \cdot n - 1 \cdot n}{(2a - 1) \cdot n} \\
 &= \frac{\sum_{i=1}^n ((1 - a) + (2a - 1) \cdot \pi) + a \cdot n - 1 \cdot n}{(2a - 1) \cdot n} \\
 &= \frac{n \cdot ((1 - a) + (2a - 1) \cdot \pi) + a \cdot n - 1 \cdot n}{(2a - 1) \cdot n} \\
 &= \frac{((1 - a) + (2a - 1) \cdot \pi) + a - 1}{(2a - 1)} \\
 &= \frac{-(a - 1) + (2a - 1) \cdot \pi + (a - 1)}{(2a - 1)} \\
 &= \frac{(2a - 1) \cdot \pi}{(2a - 1)} \\
 &= \pi .
 \end{aligned} \tag{1.12}$$

We have shown that the expected value of π_{ML} is equal to the true value of π (Equation 1.12). Therefore, we can conclude that our estimator for π_{ML} is unbiased.

1.1.6 (f)

Derive an expression for the variance of π_{ML} . Analyse its dependence on a .

We have

$$\begin{aligned}
 \mathbb{V}[\pi_{ML}] &= \mathbb{V}\left[\frac{\frac{m}{n} + a - 1}{(2a - 1)}\right] \\
 &= \mathbb{V}\left[\frac{m}{n} + a - 1\right] \cdot \frac{1}{(2a - 1)^2} \\
 &= \mathbb{V}\left[\frac{1}{n} (m + a \cdot n - 1 \cdot n)\right] \cdot \frac{1}{(2a - 1)^2} \\
 &= \mathbb{V}[m + a \cdot n - 1 \cdot n] \cdot \frac{1}{(2a - 1)^2 \cdot n^2} \\
 &= \mathbb{V}\left[\sum_{i=1}^n X_i + a \cdot n - 1 \cdot n\right] \cdot \frac{1}{(2a - 1)^2 \cdot n^2} \\
 &= \mathbb{V}\left[\sum_{i=1}^n X_i\right] \cdot \frac{1}{(2a - 1)^2 \cdot n^2} \\
 &= \sum_{i=1}^n \mathbb{V}[X_i] \cdot \frac{1}{(2a - 1)^2 \cdot n^2} .
 \end{aligned} \tag{1.13}$$

We now compute the variance of X_i . We have:

$$\begin{aligned}
 \mathbb{E}[X_i^2] &= P(X_i = 1) \cdot 1^2 + P(X_i = 0) \cdot 0^2 \\
 &= P(X_i = 1) \\
 &= (1 - a) + (2a - 1) \cdot \pi ,
 \end{aligned} \tag{1.14}$$

and

$$\begin{aligned}
 \mathbb{E}[X_i]^2 &= ((1-a) + (2a-1) \cdot \pi)^2 \\
 &= (1-a)^2 + (2a-1)^2 \cdot \pi^2 + 2 \cdot (1-a) \cdot (2a-1) \cdot \pi \\
 &= 1^2 + a^2 - 2a + 4a^2\pi^2 + \pi^2 - 4a \cdot \pi^2 + (2-2a) \cdot (2a-1) \cdot \pi \\
 &= 1 + a^2 - 2a + 4a^2\pi^2 + \pi^2 - 4a \cdot \pi^2 + (4a-2-4a^2+2a) \cdot \pi \\
 &= 1 + \pi(6a-4a^2-2) + \pi^2(4a^2-4a+1) - 2a + a^2.
 \end{aligned} \tag{1.15}$$

Therefore, the variance of X_i is given by

$$\begin{aligned}
 V(X_i) &= \mathbb{E}[X_i^2] - E[X_i]^2 \\
 &= \pi(2a-1) + 1-a - (1 + \pi(6a-4a^2-2) + \pi^2(4a^2-4a+1) - 2a + a^2) \\
 &= \pi(2a-1) + 1-a-1 - \pi(6a-4a^2-2) - \pi^2(4a^2-4a+1) + 2a - a^2 \\
 &= \pi(2a-1) - \pi(6a-4a^2-2) - \pi^2(4a^2-4a+1) + a - a^2 \\
 &= \pi(2a-1 - (6a-4a^2-2)) - \pi^2(4a^2-4a+1) + a - a^2 \\
 &= \pi(2a-1-6a+4a^2+2) - \pi^2(4a^2-4a+1) + a - a^2 \\
 &= \pi(1-4a+4a^2) - \pi^2(1-4a+4a^2) + a - a^2 \\
 &= (1-4a+4a^2) \cdot (\pi - \pi^2) + a - a^2.
 \end{aligned} \tag{1.16}$$

Finally, to compute the variance of the estimator of π_{ML} , we plug Equation 1.16 into Equation 1.13 and get

$$\begin{aligned}
 \mathbb{V}[\pi_{ML}] &= \sum_{i=1}^n \mathbb{V}[X_i] \cdot \frac{1}{(2a-1)^2 \cdot n^2} \\
 &= \sum_{i=1}^n ((1-4a+4a^2) \cdot (\pi - \pi^2) + a - a^2) \cdot \frac{1}{(2a-1)^2 \cdot n^2} \\
 &= n \cdot ((1-4a+4a^2) \cdot (\pi - \pi^2) + a - a^2) \cdot \frac{1}{(2a-1)^2 \cdot n^2} \\
 &= ((1-4a+4a^2) \cdot (\pi - \pi^2) + a - a^2) \cdot \frac{1}{(2a-1)^2 \cdot n} \\
 &= ((2a-1)^2 \cdot (\pi - \pi^2) + a - a^2) \cdot \frac{1}{(2a-1)^2 \cdot n} \\
 &= \frac{(2a-1)^2 \cdot (\pi - \pi^2)}{(2a-1)^2 \cdot n} + \frac{a - a^2}{(2a-1)^2 \cdot n} \\
 &= \frac{\pi - \pi^2}{n} + \frac{a - a^2}{(2a-1)^2 \cdot n}.
 \end{aligned} \tag{1.17}$$

We correctly observe that the variance is undefined for $a = 0.5$. Furthermore, we observe that the variance decreases as n increases, which is to be expected as we get more evidence with more data. Lastly, we observe that whenever a approaches either 0 or 1, the variance decreases. The same phenomenon is observed for the true value of π .

Figure 1.1 shows the log-scaled variance of π_{ML} as a function of a and π , which clearly displays the behavior previously described. In short, our monte-carlo simulation confirms that the variance of the estimator of π_{ML} is maximal whenever a approaches 0.5, and furthermore increased as π approaches 0.5. This makes sense, as whenever $a = 0.5$ (or close), there is very little information gained from the answer of the users, a yes could be from both groups. Hence, the variance of the estimator will be very large in this case. Furthermore, whenever one group is much larger than the other (i.e., $\pi \rightarrow 0$ or $\pi \rightarrow 1$), we expect the answers to be more consistent, and hence the estimation of π will be more accurate. Thus, the variance decreases as π approaches the extremes of 0 and 1.

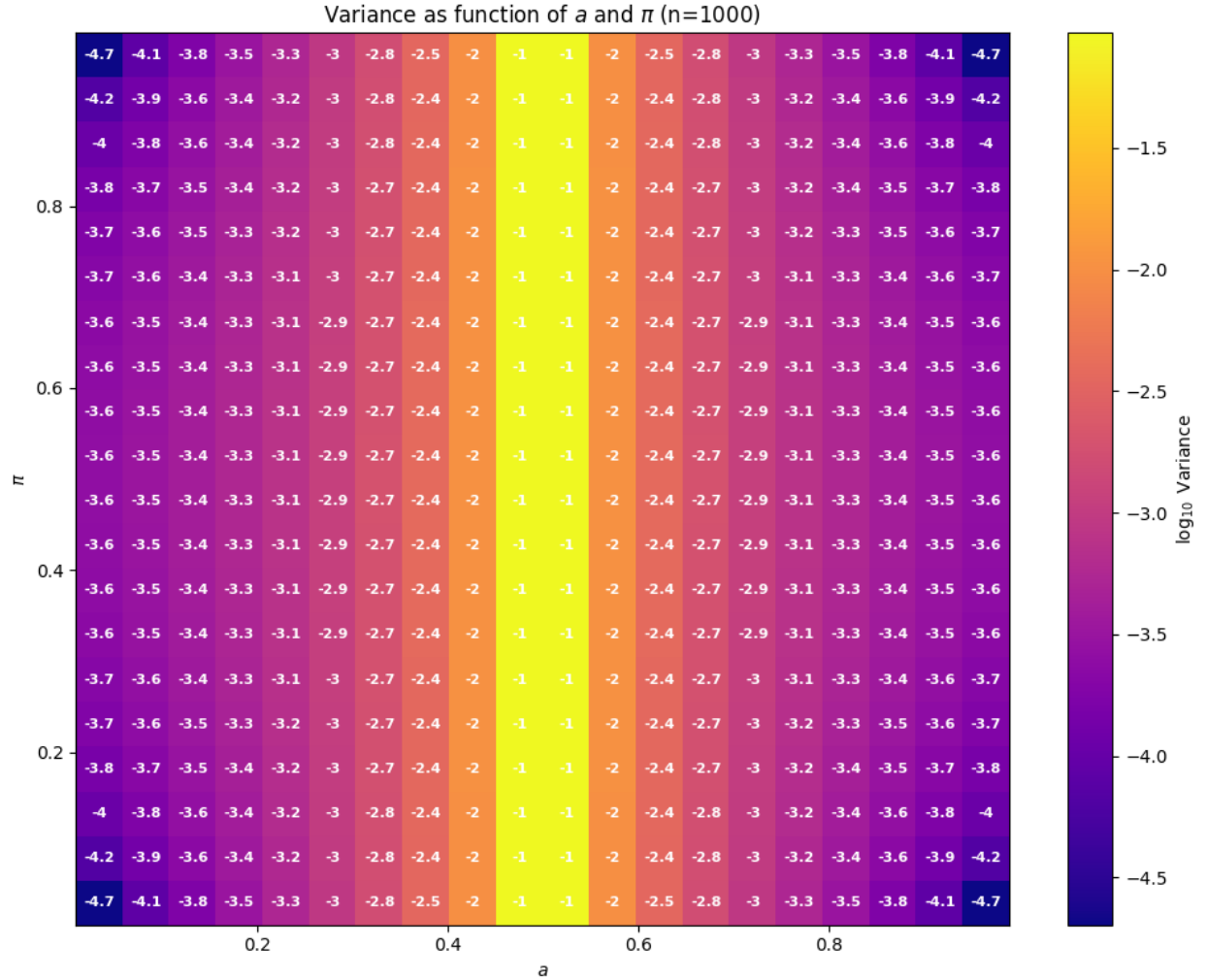


Figure 1.1: Variance of π_{ML} as function of a and π for different values of n . Here, the values are log-scaled for better display. Note that we restrict $a \in [0.01, 0.99]$ and $\pi \in [0.01, 0.99]$ for the plot. Hence, very low values are expected whenever a or π exceeds these bounds.

1.1.7 (g)

Give an expression for the bias of this naive estimator. Is it asymptotically unbiased?

We aim to recover the value of π from direct questioning. However, persons belonging to the sensitive group will lie with probability ℓ , whereas persons belonging to the non-sensitive group will answer truthfully and therefore lie with probability 0.

Naively, we could estimate the value of π as the fraction of persons answering yes to the question:

$$\pi_{\text{naive}} = \frac{\sum_{i=1}^n X_i}{n}.$$

We can model the whether a person belongs to the sensitive group as a hidden variable $S_i \in \{0, 1\}$ with $S_i = 1$ describing that the person belongs to the sensitive group, and $S_i = 0$ describes that a person belongs to the non-sensitive group.

Naturally, we have that

$$\begin{aligned} P(S_i = 1) &= \pi \\ P(S_i = 0) &= 1 - \pi \end{aligned}$$

which is distributed according to a Bernoulli distribution with parameter $p = \pi$.

We can now express the probability of a person answering yes or no to the question by marginalizing over the hidden variable S_i :

$$\begin{aligned} P(X_i = 1) &= P(X_i = 1, S_i = 1) + P(X_i = 1, S_i = 0) \\ &= \pi \cdot (1 - \ell) + 0 \\ P(X_i = 0) &= P(X_i = 0, S_i = 1) + P(X_i = 0, S_i = 0) \\ &= \pi \cdot \ell + (1 - \pi). \end{aligned}$$

We verify that the probabilities are correct by adding both $P(X_i = 1)$ and $P(X_i = 0)$ and getting 1:

$$\begin{aligned} P(X_i = 1) + P(X_i = 0) &= \pi \cdot (1 - \ell) + \pi \cdot \ell + (1 - \pi) \\ &= \pi(1 - \ell + \ell) + 1 - \pi \\ &= \pi(1) + 1 - \pi \\ &= 1. \end{aligned}$$

We now compute the expected value of the naive estimator of π to check whether the estimator is unbiased.

We have

$$\begin{aligned}
 \mathbb{E}[\pi_{\text{naive}}] &= \mathbb{E}\left[\frac{\sum_{i=1}^n X_i}{n}\right] \\
 &= \frac{1}{n} \cdot \sum_{i=1}^n \mathbb{E}[X_i] \\
 &= \frac{1}{n} \cdot \sum_{i=1}^n (P(X_i = 1) \cdot 1 + P(X_i = 0) \cdot 0) \\
 &= \frac{1}{n} \cdot \sum_{i=1}^n (P(X_i = 1)) \\
 &= \frac{1}{n} \cdot \sum_{i=1}^n (\pi \cdot (1 - \ell)) \\
 &= \frac{1}{n} \cdot n (\pi \cdot (1 - \ell)) \\
 &= \pi \cdot (1 - \ell) .
 \end{aligned} \tag{1.18}$$

We can now use the expected value of π_{naive} to compute the bias of the estimator:

$$\begin{aligned}
 \text{Bias}(\pi_{\text{naive}}, \pi) &= [\mathbb{E}[\pi_{\text{naive}}] - \pi]^2 \\
 &= [\pi \cdot (1 - \ell) - \pi]^2 \\
 &= [\pi - \pi\ell - \pi]^2 \\
 &= [-\pi\ell]^2 \\
 &= \pi^2 \ell^2 .
 \end{aligned} \tag{1.19}$$

Therefore, only whenever $\ell = 0$, i.e., no one lies, is the naive estimator of π unbiased. Otherwise, the estimator is biased towards and goes towards zero bias whenever either ℓ or π goes to zero.

1.1.8 (h)

Derive expressions for the mean squared error (MSE) of the randomized response estimator and the naive estimator. Analyse the expressions to draw conclusions about when, depending on the relevant parameters (π, a, ℓ, n) , one estimator should be favored over the other.

We aim to compute the mean squared error of our two estimators of π .

We use the Bias Variance decomposition to decompose the MSE into the bias and variance terms.² Therefore,

²See https://en.wikipedia.org/wiki/Mean_squared_error#Proof_of_variance_and_bias_relationship.

we simplify the MSE into

$$\begin{aligned}\text{MSE}(\hat{\pi}, \pi) &= \mathbb{E} \left[(\hat{\pi} - \mathbb{E}[\hat{\pi}])^2 \right] + [\mathbb{E}[\hat{\pi}] - \pi]^2 \\ &= \mathbb{V}[\hat{\pi}] + [\mathbb{E}[\hat{\pi}] - \pi]^2 \\ &= \mathbb{V}[\hat{\pi}] + \text{Bias}(\hat{\pi}, \pi)\end{aligned}$$

Therefore, we have

$$\begin{aligned}\mathbb{V}[\pi_{\text{naive}}] &= \mathbb{V} \left[\frac{\sum_{i=1}^n X_i}{n} \right] \\ &= \frac{1}{n^2} \sum_{i=1}^n \mathbb{V}[X_i] \\ &= \frac{1}{n^2} \sum_{i=1}^n [\mathbb{E}[X_i^2] - \mathbb{E}[X_i]^2] \\ &= \frac{1}{n^2} \sum_{i=1}^n [P(X_i = 1)] - (P(X_i = 1))^2 \\ &= \frac{1}{n^2} \sum_{i=1}^n (\pi \cdot (1 - \ell)) - (\pi \cdot (1 - \ell))^2 \\ &= \frac{n}{n^2} \left((\pi \cdot (1 - \ell)) - (\pi \cdot (1 - \ell))^2 \right) \\ &= \frac{\pi \cdot (1 - \ell) - \pi^2 \cdot (1 - \ell)^2}{n}\end{aligned}\tag{1.20}$$

$$\begin{aligned}\text{MSE}(\pi_{\text{naive}}, \pi) &= \mathbb{V}[\pi_{\text{naive}}] + \text{Bias}(\hat{\pi}, \pi) \\ &\stackrel{(1.20)}{=} \frac{\pi \cdot (1 - \ell) - \pi^2 \cdot (1 - \ell)^2}{n} + \text{Bias}(\hat{\pi}, \pi) \\ &\stackrel{(1.19)}{=} \frac{\pi \cdot (1 - \ell) - \pi^2 \cdot (1 - \ell)^2}{n} + \pi^2 \ell^2.\end{aligned}$$

Because the π_{ML} is an unbiased estimator of π , we have that $\text{Bias}(\pi_{ML}, \pi) = 0$. Therefore, we have

$$\begin{aligned}\text{MSE}(\pi_{ML}, \pi) &= \mathbb{V}[\pi_{ML}] + \text{Bias}(\pi_{ML}, \pi) \\ &= \mathbb{V}[\pi_{ML}] \\ &\stackrel{(1.17)}{=} \frac{\pi - \pi^2}{n} + \frac{a - a^2}{(2a - 1)^2 \cdot n}.\end{aligned}$$

First, we observe that whenever no-one lies ($\ell = 0$), the left side of the MSE of π_{naive} is equal to the left side of the MSE of π_{ML} . However, as the right side of the MSE of π_{naive} is multiplied with ℓ^2 , the MSE is

zero for $\ell = 0$, whereas the MSE of π_{ML} is non-zero, given $a \notin \{0, 1\}$. If $a \in \{0, 1\}$, then both estimators are equal w.r.t. the MSE.

Due to the complex expressions, finding the cut-off points for when both MSE expressions are equal — and therefore finding whenever one estimator is better than the other analytically — is very complicated. While we have explained the trivial edge cases, whenever we have $a \notin \{0, 1\} \wedge \pi \notin \{0, 1\}$, we have no easy way to determine when which estimator is better than the other. To overcome this issue, we have computed the expected MSE for both estimators for a range of values for a, π, ℓ , and n and show the results in Figure 1.2. Figure 1.2 clearly shows the complex, non-linear relationship between both MSE expression.

First, it is clear that no single best estimator exists for all parameters. The shape of the region where π_{naive} has a lower MSE than π_{ML} is quite clear: whenever $\ell = 0$, the naive estimator is better than the ML estimator. Furthermore, whenever $\pi = 0$, the naive estimator is also better. Both cases make sense, as whenever no one lies, the naive estimator is the better estimator of the two.

Figure 1.2 also shows that whenever a is close or equal to 0.5, the naive estimator is the better solution. Note that while our plot shows that in the subplot (d), for some parameters which include $a = 0.5$, the ML estimator is to be favored. This is not true, as it is simply an artefact of the plotting; We use only 100 points for each axis, and therefore the voxels are quite large. With increased resolution, the same plot would show that the naive estimator is the better solution for these parameters.

Lastly, for all other cases that we did not mention, the ML estimator is the better solution for large n . Refer to Figure 1.2 for the exact shape. This is logical, as the MSE of the naive estimator has a part independent of n , whereas the MSE of the ML estimator is entirely divided by n . Thus, for large n , the ML estimator will go towards zero, whereas the naive estimator will remain constant.

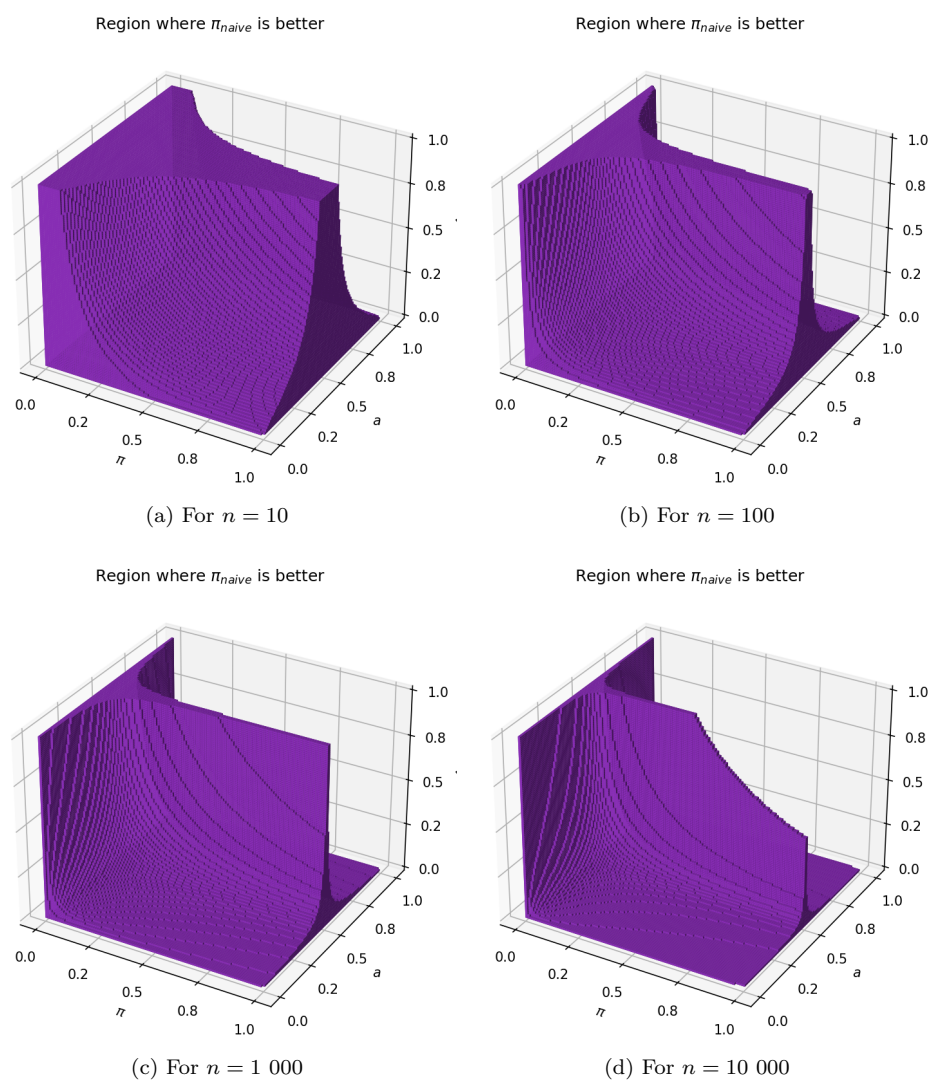


Figure 1.2: Plots of the parameter space of (π, a, ℓ) for various sample sizes n . The purple voxels indicate for which parameters the MSE of π_{naive} is lower than the MSE of π_{ML} . Note that plot is mirrored around the ℓ -axis at $a = 0.5$.

1.2 Problem 2

We have implemented the given formulas for estimating the population size $\hat{N}$ and its variance $\hat{\sigma}_{\hat{N}}$ given some probability distributions from which we sample the probability of catching one animal. We have run the Monte-Carlo simulations for 1000 runs for each given probability distribution and report on the computed values in Table 1.1 for $p_i \sim U(0, 0.01)$, Table 1.2 for $p_i \sim U(0, 0.02)$, Table A.1 for $p_i \sim 0.02 \cdot \text{Beta}(1, 3)$, and Table A.2 for $p_i \sim 0.01 \cdot \text{Beta}(2, 2)$.

Note that for the sake of ease of comparison, we report on $\hat{\sigma}_{\hat{N}} = \sqrt{\hat{\sigma}_{\hat{N}}^2}$ instead of the variance $\hat{\sigma}_{\hat{N}}^2$ as it allows us to directly compare it with the reported standard deviations of the other parameters.

In this question, we are interested in the differences that arise from the different probability distributions, besides their performances.

We first plot the probability distribution for the four probability distributions that we are investigating in Figure 1.3. From that figure, we hypothesize that the expected value of catching one animal is not the same for all four probability distributions. Therefore, let us first compute the expected value of catching one animal for each probability distribution.

For the uniform distribution $U(0, a)$, with $a \in \mathbb{R} \setminus \{0\}$, we have:

$$\begin{aligned}
 \mathbb{E}_{p \sim U(0, a)}[X_i^t] &= \mathbb{E}_{p \sim U(0, a), q \sim U(0, 1)}[q \leq p] \\
 &= \mathbb{E}_{p \sim U(0, a)}[\mathbb{E}_{q \sim U(0, 1)}[q \leq p]] \\
 &= \int_0^a f_p(p) \cdot \mathbb{E}_{q \sim U(0, 1)}[q \leq p] \, dp \\
 &= \int_0^a \frac{1}{a} \cdot \mathbb{E}_{q \sim U(0, 1)}[q \leq p] \, dp \\
 &= \frac{1}{a} \int_0^a \mathbb{E}_{q \sim U(0, 1)}[q \leq p] \, dp \\
 &= \frac{1}{a} \int_0^a \int_0^1 f_q(q) \cdot \mathbb{1}\{q \leq p\} \, dq \, dp \\
 &= \frac{1}{a} \int_0^a \int_0^1 1 \cdot \mathbb{1}\{q \leq p\} \, dq \, dp \\
 &= \frac{1}{a} \int_0^a \int_0^p 1 \, dq \, dp \\
 &= \frac{1}{a} \int_0^a [q]_0^p \, dp \\
 &= \frac{1}{a} \int_0^a p \, dp \\
 &= \frac{1}{a} \left[\frac{p^2}{2} \right]_0^a \\
 &= \frac{1}{a} \frac{a^2}{2} \\
 &= \frac{a}{2}.
 \end{aligned}$$

It naturally follows that

$$\mathbb{E}_{p \sim U(0,0.01)}[X_i^t] = 0.005 \quad (1.21)$$

and

$$\mathbb{E}_{p \sim U(0,0.02)}[X_i^t] = 0.01 . \quad (1.22)$$

For $p \sim \text{Beta}(\cdot)$, we have:

$$\begin{aligned} \mathbb{E}_{p \sim \text{Beta}(\alpha, \beta)}[X_i^t] &= \mathbb{E}_{p \sim \text{Beta}(\alpha, \beta), q \sim U(0,1)}[q \leq p] \\ &= \mathbb{E}_{p \sim \text{Beta}(\alpha, \beta)}[\mathbb{E}_{q \sim U(0,1)}[q \leq p]] \\ &= \int_0^1 f_{\text{Beta}(\alpha, \beta)}(p) \cdot \mathbb{E}_{q \sim U(0,1)}[q \leq p] \, dp \\ &= \int_0^1 f_{\text{Beta}(\alpha, \beta)}(p) \cdot \int_0^1 f_q(q) \cdot \mathbb{1}\{q \leq p\} \, dq \, dp \\ &= \int_0^1 f_{\text{Beta}(\alpha, \beta)}(p) \cdot p \, dp \\ &= \mathbb{E}_{X \sim \text{Beta}(\alpha, \beta)}[X] \\ &= \frac{\alpha}{\alpha + \beta} . \end{aligned} \quad (1.23)$$

Note that if $p \sim a \cdot \text{Beta}(\cdot)$ with $a \in \mathbb{R} \setminus \{0\}$, we can just multiply Equation 1.23 with a due to the properties of the expectation and the integrals. a would be contained in the density function as a multiplicative factor, and thus, we can extract it. Note that I got the expectation of the Beta distribution from wikipedia.³

Therefore, it follows that

$$\mathbb{E}_{p \sim 0.02 \cdot \text{Beta}(1,3)}[X_i^t] = 0.02 \cdot \frac{1}{4} = 0.005 \quad (1.24)$$

and

$$\mathbb{E}_{p \sim 0.01 \cdot \text{Beta}(2,2)}[X_i^t] = 0.01 \cdot \frac{1}{2} = 0.005 . \quad (1.25)$$

The usefulness of Equation 1.21–1.22 and Equation 1.24–1.25 expected values will become clear in the next section.

Expected Behavior. Before discussing the results of the Monte-Carlo simulations in detail, let us first investigate the expected behavior of the simulations.

We have computed the expected number of times an animal is being caught at each time step. Recall that the expected values for X_i^t is 0.005 for all given distributions, except when $p \sim U(0,0.01)$, as in that setting, the expected value rises to 0.01.

Concretely, this means that when using $U(0,0.01)$ as the probability to catch an animal, the number of times an animal is being caught is twice as high as when using the three other probability distribution. Therefore, we expect the estimations of $\hat{N}$ and $\sigma_{\hat{N}}$ to be better when using $p \sim U(0,0.01)$ as opposed to the other distributions.

³https://en.wikipedia.org/wiki/Beta_distribution

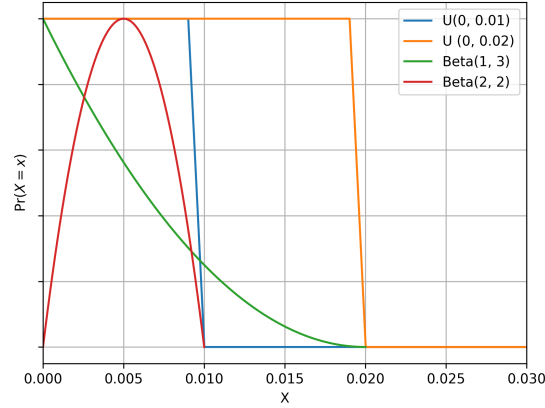


Figure 1.3: Probability density functions for the four different probability distributions. We normalize the probability density functions to have a maximum value of 1 for each distribution as we are interested in the relative differences between the different distributions. While the height of the pdfs are not to scale, their shape are.

Trivially, as the expected behavior of the three other distributions are identical, we expect the estimations of $\hat{N}$ and $\sigma_{\hat{N}}$ to be very identical up to the effect of the randomness introduced by the Monte-Carlo simulations.

Observations. Let us first look at Table 1.1 which contains the result when using $p \sim U(0, 0.01)$.

First, it is very clear that increasing the true population size N decreases the relative error of the estimator. Furthermore, the estimated population size $\hat{N}$ is close to the true value of N for all three settings of our simulation with a deviation of approximately 90–150 animals.

Interestingly, although the absolute error of the population estimation increases with the true number of population, the relative error of the estimated population size to the true population size decreases with increasing true population size N . Whereas for a small population size ($N = 500$), the relative error is approximately 18%, for larger population sizes ($N = 5000$), the relative error drops to only 3%.

The estimated standard deviation ($\hat{\sigma}_{\hat{N}} = \sqrt{\hat{\sigma}_{\hat{N}}^2}$) from the given formula perfectly matches the standard deviation computed by the Monte-Carlo simulation (refer to the gray value next to the $\hat{N}$ estimate). Nevertheless, it is important to denote that the standard deviation of the estimated variance is very large (e.g., $\sigma_{\hat{\sigma}_{\hat{N}}} = \pm 258$ for $\hat{\sigma}_{\hat{N}} = 239.58$), hence, the estimated variance can deviate significantly across different runs. Therefore, the estimated variance is only trustworthy for large population sizes.

It is worth mentioning that although the standard deviation increases with increased population size N , the variance of the estimated standard deviation of the estimation of the population size decreases the larger N becomes.

We compute the coverage (percentage of times the true population size is contained in the confidence interval) for different values of $\alpha \in \{0.05, 0.01\}$ and observe that, regardless of the true population size N , the average coverage matches the expected coverage of the specified confidence intervals.

Interestingly, the standard deviation of the computed coverage from the Monte-Carlo simulation remains

$$p_i \sim U(0, 0.01)$$

		$N = 500$	$N = 1000$	$N = 5000$
$\hat{N}$		590.45±311	1096.62±286	5152.14±583
Error (%)		18.09	9.66	3.04
$\hat{\sigma}_{\hat{N}}$		239.58±258	284.17±116	566.36±98
C.I.	$\alpha = 0.05$	[299.68±85; 1322.77±1298]	[687.34±137; 1842.05±609]	[4179.35±423; 6414.78±804]
	$\alpha = 0.01$	[250.36±59; 1740.30±2079]	[601.62±110; 2185.30±773]	[3922.46±383; 6884.52±890]
Coverage (%)	$\alpha = 0.05$	94.50±22	95.70±20	94.70±22
	$\alpha = 0.01$	99.40±7	99.20±8	99.20±8
Bias ($\approx$)		8 181	9 334	23 147
Variance ($\approx$)		124 464	94 211	330 364
MSE ($\approx$)		132 646	103 546	353 512

Table 1.1: Result from 1000 MC simulations for estimating $\hat{N} \mid p_i \sim U(0, 0.01)$. For each value, we report on the mean (black) and standard deviation (gray). We omit the digits after the decimal point for the standard deviation for brevity. The Error (%) is the absolute difference between the estimated and the true value of N , divided by the true value of N . The Bias is reported as $[\mathbb{E}[X] - X]^2$.

stable across all population sizes; We have a standard deviation of ± 22 percentage points for $\alpha = 0.05$ and ± 8 percentage points for $\alpha = 0.01$.

Finally, and most interestingly, we also compute the Bias, Variance, and the Mean Squared Error (MSE) for each context of our simulations. In Table 1.1, we observe that the bias only increases slightly from $N = 500 \rightarrow N = 1000$ (from $90^2 \rightarrow 96^2$) but increases drastically for $N = 1000 \rightarrow N = 5000$, where the bias increases by 2.5 times.

Most interestingly, the variance does not follow a simple pattern. While increasing $N = 500 \rightarrow N = 1000$ drops the variance by 23.7% (around 30 000 units of absolute difference), when increasing the true population size from $N = 1000 \rightarrow N = 5000$, we observe an increase of the variance of the estimation of the population size 350%.

Finally, because the variance is around 1 order of magnitude larger than the bias term, the MSE follows the general trend described in the variance.

Comparison to other Probability Distributions. While we went into details on the results from Table 1.1, these results result from $p \sim U(0, 0.01)$ and may therefore deviate from the results for the other probability distributions.

First, as hypothesized previously, the results between

- $p \sim U(0, 0.01)$ - See Table 1.1
- $p \sim 0.02 \cdot \text{Beta}(1, 3)$ - See Table A.1
- $p \sim 0.01 \cdot \text{Beta}(2, 2)$ - See Table A.2

are extremely similar. In fact, not only are the returned values all within the standard deviation of the results from each other — regardless of the true population size or the metric. Naturally, as each distribution has

$$p_i \sim U(0, 0.02)$$

		$N = 500$	$N = 1000$	$N = 5000$
$\hat{N}$		527.70±98	1035.16±134	5088.17±284
Error (%)		5.54	3.52	1.76
$\hat{\sigma}_{\hat{N}}$		97.71±29	133.04±27	288.03±25
C.I.	$\alpha = 0.05$	[381.03±57; 774.58±172]	[818.85±91; 1347.52±198]	[4568.45±240; 5700.59±337]
	$\alpha = 0.01$	[348.62±49; 882.73±206]	[765.73±82; 1471.85±225]	[4421.98±228; 5914.73±356]
Coverage (%)	$\alpha = 0.05$	95.00±21	95.30±21	94.60±22
	$\alpha = 0.01$	99.40±7	98.80±10	98.70±11
Bias ($\approx$)		767	1 236	7 774
Variance ($\approx$)		10 398	18 460	83 612
MSE ($\approx$)		11 166	19 696	91 387

Table 1.2: Result from 1000 MC simulations for estimating $\hat{N} \mid p_i \sim U(0, 0.02)$. For each value, we report on the mean (black) and standard deviation (gray). We omit the digits after the decimal point for the standard deviation for brevity. The Error (%) is the absolute difference between the estimated and the true value of N , divided by the true value of N . The Bias is reported as $[\mathbb{E}[X] - X]^2$.

a different probability density function, and that Monte-Carlo simulation inherently introduces some noise, the results slightly differs between Table 1.1, Table A.1, and Table A.2.

However, when we look at Table 1.2, which contains the results for $p \sim U(0, 0.02)$, which has a higher expected value (see Equation 1.22), we observe that, indeed, the values differ (significantly) from the results of the other distributions.

First, the deviation of the estimated to the true population size is significantly lower for $p \sim U(0, 0.02)$ compared to the other distributions. The relative error is the perfect metric to quantify the change in deviation, as it decreases from 18% for $p \sim U(0, 0.01)$ to 5.54% for $p \sim U(0, 0.02)$ when $N = 500$, which is a decrease of 70%. Increasing the true population size to larger values has a similar effect. For example, for $N = 5000$, the relative error drops from 3% to 1.76%.

We remark that the estimated standard deviation and the true standard deviation of the estimated population size are very similar for $p \sim U(0, 0.02)$ too, with a lower standard deviation compared to the other distributions.

Similarly, the coverage remains stable across all population sizes, demonstrating that the confidence intervals are well-calibrated. Note that the standard deviation of the coverage is the same as for the other distributions, indicating that the std. of the coverage is potentially independent of the probability distribution used.

Finally, the MSE follows a different pattern as computed for the other distributions. While the bias increases with the same pattern but lower values as the other distributions, the variance also increases linearly with the size of the true population size. Therefore, the MSE roughly follows a linear increase with the size of the true population size.

However, the increase in the quality of the estimators is significant, as the MSE for $N = 5000$ drops from around 350 000 units to around 90 000 units, which is a decrease of around 75%.

Discussion — MSE. One of the most surprising observations is the change of the MSE w.r.t. the true

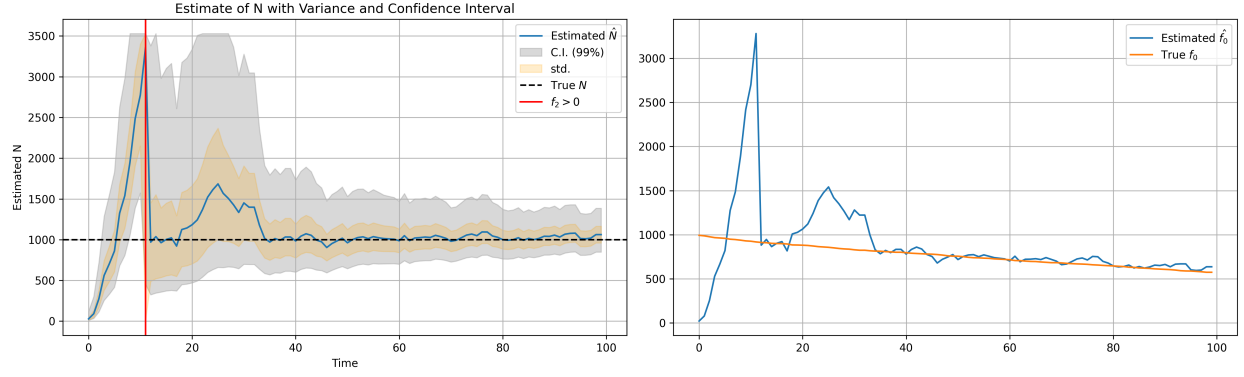


Figure 1.4: Single run of the Monte-Carlo simulation for estimating $\hat{N}$ with a true population size of $N = 1000$ and a probability distribution of $p_i \sim U(0, 0.01)$.

population size. The results from Table 1.1 and Table A.2 show that the MSE (and variance) follows a v shape when increasing the true population size, where $N = 1000$ has the lowest MSE when comparing to $N = 500$ and $N = 5000$.

We hypothesize that this phenomenon is simply due to the fact that the number of steps ($T = 40$) is too low to obtain accurate estimations when using probability distributions that have a low $\mathbb{E}[X_i^t = 1]$. We hypothesize that doubling the number of time steps should give us similar results as the results from Table 1.2 with $p \sim U(0, 0.02)$.

This hypothesis is supported by the results in Table 1.2 where the MSE follows a linear increase w.r.t. N .

To verify our hypothesis, we run the simulation using $p \sim U(0, 0.01)$ for twice as long, namely $T = 80$ instead of 40 time steps. We report on the results in Table A.3 in the Appendix. In short, we observe that the results from Table A.3 matches the results obtained in Table 1.2, therefore validating our hypothesis.

Discussion — Estimated Population Size. Interestingly, the estimated population size reported by our Monte-Carlo simulations always over-estimate the true population size. We want to understand this better, and understand whether this is a systematic property of the given estimator.

First, we hypothesize that most of the variance is due to the estimation of $\hat{f}_0$. As the given formula for estimating $\hat{f}_0$ is dependent on whether $f_2 \neq 0$ or $f_2 = 0$, we aim to first understand how similar both formulas are. To this end, we plot in Figure 1.4 the estimated $\hat{N}$ over 100 time steps for a single run of the Monte-Carlo simulation with true population size $N = 1000$ and $p \sim U(0, 0.01)$.

In Figure 1.4, we clearly observe that, whenever $f_2 = 0 \rightarrow f_2 > 0$, the estimation of $\hat{f}_0$ is not continuous in time and drastically drops. Interestingly, as soon as $f_2 > 0$, the estimator of $\hat{N}$ seems to approach the true population size.

We also plot the true value for f_0 and the estimated value for $\hat{f}_0$ over the same number of time steps in Figure 1.4. We can clearly see that the estimated values for $\hat{f}_0$ approach the true value for f_0 over time.

We remark that the behavior observed in Figure 1.4 is not systematic, as the jump is very extreme. We found this example by chance when plotting the results for a single run. However, it provides us with great insights into the behavior of the estimators.

End of the core answer of Question 2

While this remaining part relates to the discussion of Problem 2, and can therefore be considered as part of the answer, we understand that only readers interested in the problem formulated in Equation 1.26 will be interested in this section. Therefore, if the reader thinks that the explanation stated above is sufficient, we do not force them to read the remaining part.

Analytical Analysis — Over-Estimation of the Population Size.

In the remaining part of this section, we will investigate analytically whether the over-estimation of the population size is a systematic property of the given estimator.

Formally, we are interested in the following question

$$\Pr(\hat{N} > N) > \Pr(\hat{N} \leq N) . \quad (1.26)$$

To this end, we will assume that the estimator is unbiased, i.e., that $\mathbb{E}[\hat{N}] = N$. Note that Equation 1.26 is not in contradiction with the unbiasedness of the estimator, as the expected value is the value towards which the estimator converges to in the limit of the number of samples. However, for finite samples, even if the estimator has a theoretical mean of N , if the distribution of the estimated population size is skewed, it is possible that Equation 1.26 holds.

Before we start, let us first formally define some notation and definitions of f_i that allows us to derive the probability for $\hat{N} > N$.

Let us denote an animal as X_i with $i \in \{1, \dots, N\}$ and $X_i^t \in \{0, 1\}$ denoting whether the animal i has been caught at time step t . Therefore, we can define

$$X_i = \sum_{t=1}^T X_i^t ,$$

i.e., the total number of times the animal has been caught after T time steps.

Logically, we have that $X_i^t \sim \text{Bernoulli}(p)$ with p being the probability of being caught at time step t . In our setting, p follows a specified probability distribution.

We have that X_i^t are i.i.d. w.r.t. the time steps t and the animals i . These are important assumptions. From this, it follows that $X_i \sim \text{Binomial}(T, p)$ as we have a sum of T independent Bernoulli variables.

We can now formally define f_k as follows using the following set

$$S_k = \{X_i : i \in \{1, \dots, N\} \mid X_i = k\}$$

which contains all the animals that have been caught exactly k times within the T time steps. Therefore, f_k can be defined as follows:

$$f_k = |S_k| .$$

Because f_k is a random variable, we can define the probability distribution of f_k w.r.t. the number of time

steps t as binomial distribution:

$$\Pr(f_k = n \mid T = t) = \text{Binomial}(N, n, \Pr(X_i = k \mid T = t)) \quad n \in \{0, 1, \dots, N\}$$

Therefore, we need to compute the probability of an animal being caught exactly k times within the T time steps. We have

$$\Pr(X_i = k \mid T = t') = \text{Binomial}(t', k, \Pr(X_i^t = 1))$$

where $\Pr(X_i^t = 1)$ is the probability of an animal being caught at time step t . Because X_i^t is a binary variable, its expected value is its probability, and we can therefore re-use the expected values for the different given probabilities that we computed earlier (see Equation 1.21, 1.22, 1.24, 1.25).

We can now compute the expected value for each f_k :

$$\begin{aligned} \mathbb{E}[f_k \mid T = t] &= \sum_{n=0}^N n \cdot \Pr(f_k = n \mid T = t) \\ &= \sum_{n=1}^N n \cdot \text{Binomial}(N, n, \Pr(X_i = k \mid T = t)) \\ &= \sum_{n=1}^N n \cdot \text{Binomial}(N, n, \text{Binomial}(t, k, p)) \\ &= \sum_{n=1}^N n \cdot \text{Binomial}\left(N, n, \binom{t}{k} p^k (1-p)^{t-k}\right) \\ &= \mathbb{E}\left[\text{Binomial}\left(N, n, \binom{t}{k} p^k (1-p)^{t-k}\right)\right] \\ &= N \cdot \binom{t}{k} p^k (1-p)^{t-k} \end{aligned}$$

	Expected Value for n		
N	500	1000	5000
$U(0, 0.01)$	90.83	181.67	908.39
$U(0, 0.02)$	165.51	331.02	1655.14

Table 1.3: Expected value for n for $T = 40$ and $N \in \{500, 1000, 5000\}$ for the different probability distributions.

Therefore, we can now compute the expected value for n :

$$\begin{aligned}
 \mathbb{E}[n] &= \mathbb{E}\left[\sum_{k=1}^T f_k\right] \\
 &= \sum_{k=1}^T \mathbb{E}[f_k] \\
 &= \sum_{k=1}^T \mathbb{E}[f_k \mid T = T] \\
 &= \sum_{k=1}^T N \cdot \binom{T}{k} p^k (1-p)^{T-k} \\
 &= N \cdot \sum_{k=1}^T \binom{T}{k} p^k (1-p)^{T-k} \\
 &= N \cdot \left[\left(\sum_{k=0}^T \binom{T}{k} p^k (1-p)^{T-k} \right) - \binom{T}{0} p^0 (1-p)^{T-0} \right] \\
 &= N \cdot \left[1 - (1-p)^T \right] \\
 &= N \cdot \left(1 - (1-p)^T \right) .
 \end{aligned} \tag{1.27}$$

We report on the expected values for n with $T = 40$ for our four probability distributions in Table 1.3. Note that Equation 1.27 is equal to

$$\begin{aligned}
 \mathbb{E}[n] &= N \cdot \left(1 - (1-p)^T \right) \\
 &= N \cdot \left(1 - \binom{T}{0} p^0 (1-p)^T \right) \\
 &= N - N \cdot \binom{T}{0} p^0 (1-p)^T \\
 &= N - \mathbb{E}[f_0 \mid T = T]
 \end{aligned}$$

which is exactly what we would expect.

Because $\hat{N} = n + \hat{f}_0$, we now compute the expected value for $\hat{f}_0$. As $\hat{f}_0$ is conditional on whether $f_2 \neq 0$ or $f_2 = 0$, we need to compute the probability of $f_2 = 0$ and $f_2 \neq 0$. Because we model f_k as a binomial

	$\Pr(f_2 = 0 \mid T = 40)$		
N	500	1000	5000
$p = 0.01$	1.3×10^{-12}	1.7×10^{-24}	1.6×10^{-119}
$p = 0.005$	2.96×10^{-4}	8.77×10^{-8}	5.19×10^{-36}

Table 1.4: Probability for $f_2 = 0$ n for $T = 40$ and $N \in \{500, 1000, 5000\}$ for the different probability distributions.

	$\mathbb{E}[\hat{f}_0 \mid f_2 = 0, T = 40]$		
N	500	1000	5000
$p = 0.01$	9113.93	36492.25	913036.85
$p = 0.005$	3375.21	13514.37	338129.79

Table 1.5: Expected value for $\hat{f}_0$ conditioned on $f_2 = 0$ for $T = 40$ and $N \in \{500, 1000, 5000\}$ for different values of p .

distribution, we can easily compute the probability of $\Pr(f_2 = 0)$. We have

$$\begin{aligned}
 \Pr(f_2 = 0 \mid T = T) &= \text{Binomial}(N, 0, \Pr(X_i = 2 \mid T = T)) \\
 &= (1 - \Pr(X_i = 2 \mid T = T))^N \\
 &= (1 - \text{Binomial}(T, 2, p))^N.
 \end{aligned}$$

We report on the probabilities for our four probability distributions for p in Table 1.4. We also plot the probability changing over time in Figure A.1.

Before investigating the case where $f_2 \neq 0$, let us first investigate the bias of $\hat{f}_0 \mid f_2 = 0$. We have

$$\begin{aligned}
 \mathbb{E}[\hat{f}_0 \mid f_2 = 0, T = t] &= \mathbb{E}\left[\frac{f_1^2 - f_1}{2}\right] \\
 &= \sum_{n=0}^N \frac{n^2 - n}{2} \cdot \Pr(f_1 = n \mid T = t) \\
 &= \sum_{n=0}^N \frac{n^2 - n}{2} \cdot \text{Binomial}(N, n, \Pr(X_i = 1 \mid T = t)) \\
 &= \sum_{n=0}^N \frac{n^2 - n}{2} \cdot \text{Binomial}(N, n, \text{Binomial}(t, 1, p)) \\
 &= \sum_{n=0}^N \frac{n^2 - n}{2} \cdot \text{Binomial}(N, n, \binom{t}{1} p^1 (1-p)^{t-1})
 \end{aligned}$$

which we cannot simplify further (at least such that gives us more insights). We use python to compute the expected value for $\hat{f}_0 \mid f_2 = 0, T = 40$ for our four probability distributions and report on the results in Table 1.5. While the expected values for $\hat{f}_0 \mid f_2 = 0$ are very large, it is easy to see that when multiplied with the probability of $f_2 = 0$, we would get values that are almost zero. The interested reader can find a plot describing the rescaled expected value in Figure A.2 in the Appendix.

Let us now investigate the bias of $\hat{f}_0$ when $f_2 \neq 0$. As $\hat{f}_0$ is defined as a function of f_1 and f_2 , and that both of them are dependent,⁴ we have to model their probability jointly for a fully analytical derivation. While it is very tempting to add a independence assumption on the distributions of f_1 and f_2 , their correlation is too strong to be ignored, and the obtained results are not meaningful.⁵ We show the correlations for our probability distributions in Table A.5 in the Appendix.

The only way to compute $\mathbb{E}[\hat{f}_0 \mid f_2 > 0]$ is by marginalizing over all possible values for all f_i with $f_i \notin \{1, 2\}$, i.e.,

$$\begin{aligned} \mathbb{E}[\hat{f}_0 \mid f_2 > 0] &= \mathbb{E}\left[\frac{f_1^2}{2f_2} \mid f_2 > 0\right] \\ &= \sum_{n=0}^{N-1} \sum_{m=1}^{N-n} \frac{n^2}{2m} \cdot \Pr(f_1 = n, f_2 = m \mid T = t) \\ &= \sum_{n=0}^{N-1} \sum_{m=1}^{N-n} \frac{n^2}{2m} \cdot \left(\sum_{k_3=0}^{N-n-m} \cdots \sum_{k_N=0}^{N-n-m-\sum_{i=3}^N k_i} \Pr(f_1 = n, f_2 = m, f_3 = k_3, \dots, f_N = k_N \mid T = t) \right) \end{aligned} \quad (1.28)$$

which is computationally infeasible. Furthermore, we cannot simplify Equation 1.28, as every f_i is dependent on each f_j with $i \neq j$. Note that especially for our simulation size, using $N = 5000$ would result in a value proportional to 2^{5000} , which is a number with 1506 digits.

Therefore, without further assumptions, we are unsure whether we can show that N is unbiased — let alone skewed — in a fully analytical way.

Nevertheless, as we can model individual f_i using a binomial distribution, we can model the skewness of both f_1 and f_2 analytically. This might give us some insights into whether Equation 1.26 holds.

Skewness

Let us now investigate the skewness of $f_i \sim \text{Binomial}(T, p_i)$. It is a well known fact that the skewness of a binomial distribution is given by

$$\text{Skewness}(f_i) = \frac{(1-p) - p}{\sqrt{N \cdot p \cdot (1-p)}}.$$

⁴We attach in the Appendix (See Section A.2.1) a proof of the dependency of f_1 and f_2 generated by ChatGPT. As the proof is not critical for the continuation of this derivation, we add it to the Appendix for the interested reader. Note that while we have deleted the exact output of ChatGPT, the idea of this proof is still to be credited to ChatGPT.

⁵This was our first attempt. However, our formula estimated $\mathbb{E}[\hat{f}_0 \mid f_2 > 0, T = 40]$ to be almost zero for all probability distributions and sample sizes.

	Skewness(f_1)			Skewness(f_2)		
N	500	1000	5000	500	1000	5000
$p = 0.01$	0.0463	0.0327	0.0146	0.17799	0.12585	0.05628
$p = 0.005$	0.0809	0.0572	0.0256	0.34368	0.24302	0.10868

 Table 1.6: Skewness for f_1 and f_2 for $T = 40$ when modeled as a binomial distribution.

For f_1 , we have that

$$\begin{aligned}
 \text{Skewness}(f_1) &= \frac{(1 - \Pr(X_i = 1 \mid T = T)) - \Pr(X_i = 1 \mid T = T)}{\sqrt{N \cdot \Pr(X_i = 1 \mid T = T) \cdot (1 - \Pr(X_i = 1 \mid T = T))}} \\
 &= \frac{\left(1 - \binom{T}{1} p^1 (1-p)^{T-1}\right) - \binom{T}{1} p^1 (1-p)^{T-1}}{\sqrt{N \cdot \left(1 - \binom{T}{1} p^1 (1-p)^{T-1}\right) \cdot \binom{T}{1} p^1 (1-p)^{T-1}}} \\
 &= \frac{1 - 2 \cdot \binom{T}{1} p^1 (1-p)^{T-1}}{\sqrt{N \cdot \left(1 - \binom{T}{1} p^1 (1-p)^{T-1}\right) \cdot \binom{T}{1} p^1 (1-p)^{T-1}}}
 \end{aligned}$$

which we will compute using python. This results in the results shown in Table 1.6.

Recall that a positive value for skewness indicates that the distribution has a median and mode on the left of the mean, and therefore the majority of the values are on the left, and the distribution is tailed to the right. A negative value has the same interpretation, but flipped. We added a visualization of the interpretation of the value of skewness in Figure A.3 in the Appendix.

From Table 1.6, it is easy to see that all values are positive. Therefore, we know that f_1 and f_2 have the majority of their values to the left of the mean, i.e., have a lower value than the mean. Naturally, the more samples we have, the less skewed the distribution is, this is also reflected in the table.

We are unable to compute the skewness of n , which follows the same argument as above, namely that all the f_i are dependent on each other. While all the f_i can be modeled using a multinomial distribution, a closed form solution to the skewness of a multinomial distribution is unknown to us, we couldn't find any literature on the topic. Therefore, we assume that n is either not skewed, or very lightly skewed in the opposite direction of $\hat{f}_0$. Otherwise, our argument does not hold.

However, and this is very interesting, we can say something about the skewness of $\hat{f}_0$. When looking at the skewness of $\hat{f}_0$ in Table 1.6, we see that f_1 is less skewed than f_2 . In fact, f_2 is skewed around 4–5 times more than f_1 . This is a very important and interesting observation.

This shows that, for small and finite samples, we are more likely to sample values for f_1 and f_2 below the mean than above. The inequality in the skewness from Table 1.6 indicates that, f_2 is sampled more below the expected value of f_2 than compared to f_1 .

As we divide the less undersampled value by the more undersampled value, we argue that $\hat{f}_0$ is negatively skewed. This follows from the argument that, assuming that $\mathbb{E}[\hat{f}_0] = f_0$ holds, we would $\frac{f_1}{f_2} \approx 5$, which would mean that we overestimate f_0 .⁶ As we are squaring the f_1 value, and only increasing the value of f_2 linearly

⁶We have not used skewness in any work before. Therefore, we are unsure if this is correct. However, that this ration is

in $\hat{f}_0$, we believe that this reduces the extremeness of the skewness of $\hat{f}_0$ significantly.

Therefore, assuming that $\mathbb{E}[\hat{f}_0] = f_0$ holds, we argue that Equation 1.26 holds. How to prove this formally is unknown to us; We believe that this statement is not true in general — as in the limit, the skewness, variance, and bias becomes zero — but we argue that especially for small finite samples, the statement holds.

We tried to get some help from ChatGPT, as we hope that it can uncover some theorems unknown to us. While it gave us a negative answer on whether the statement holds, it said that if our distribution is log-concave, then $P(X < E[X]) \geq \frac{1}{2}$ holds for some random variable $X \sim G$ with G being a log-concave positively skewed distribution. As the Binomial distribution is log-concave,⁷ we *could* argue that because we hypothesize that $\frac{f_1^2}{2f_2}$ is negatively skewed, we have $P(\hat{f}_0 > f_0) \geq \frac{1}{2} \Rightarrow P(\hat{f}_0 > f_0) > P(\hat{f}_0 \leq f_0)$. But as we are unsure about the correctness of our argument, we will only mention it here.

larger than 1 is what we are trying to argue.

⁷See https://en.wikipedia.org/wiki/Logarithmically_concave_function. We are unsure if this is correct for all cases, especially ours.

Chapter 2

Part B: Data Analysis

2.1 Data pre-processing and experimental setup

We combined two Equinox datasets containing change metrics and CK/OO metrics and used the class identifier `classname` as the join key. Defect-related metadata columns were removed from the feature set, while `bugs` was retained as the target variable.

We merged the two datasets using a one-to-one inner join on `classname`. After merging, all feature columns, as well as the target, were converted to numeric values. The final dataset contains **324 classes**, **32 features**, and one target variable.

The dataset was split into a training set (67%) and a test set (33%). To preserve the distribution of bug counts, we used stratified random sampling based on the predefined bug-count bins (0, 1–2, 3–5, 6+). The stratification variable was used only for splitting and removed afterwards.

The resulting training and test sets were saved and used consistently in all analyses. Models were selected on the training set and the final selected model was evaluated on the test set.

2.1.1 Data Analysis

Before training our Poisson and logistic regression models, we first investigate the distribution of the data.

As a first step, we plot the histogram of the 32 features that we use in Figure A.4. While we only display the marginal distributions of the features, and therefore omit the joint interaction between multiple features, plotting the joint interaction would require 512 plots, which, for the scope of this task, is too much to look at.

Figure A.4 clearly shows that most of the data are not only skewed to the left, i.e., have most of their value close to zero with heavy tails containing high values, but we also remark that the range of certain variables is extremely large (e.g. `numberOfLinesOfCode` which spans the range of around 0–1500).

While the skewness of the response variable is not an issue, as we use Poisson regression, which can handle



Figure 2.1: Histograms of the 32 features after transforming them into the log domain and normalizing them.

skewed data, and logistic regression for the binary case, we argue that the skewness in the features might impact the model negatively. Therefore, we propose to transform the 32 features into the log-domain, which should reduce the effect of the skewness.

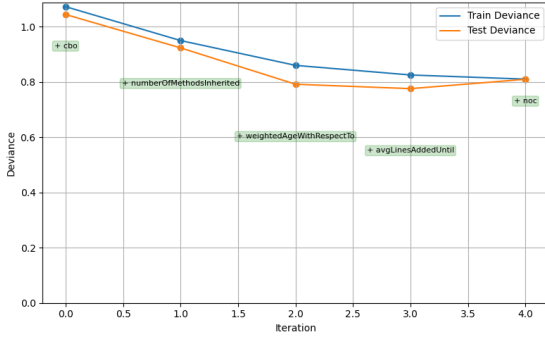
Finally, to make the training of the models easier, we normalize every feature such that they have a mean of zero and a standard deviation of 1. While this hinders slightly the interpretation of the learned parameter β , we remark that by using the inverse of the normalization used on the features, we can give meaning to the learned parameters again.

We plot the distribution of the log-transformed and normalized features in Figure 2.1. While we observe that many features remain skewed to the left, we also observe that the values of the features are more evenly distributed, with many features almost containing a gaussian bell shape (e.g. `avgCodeChurnUntil`).

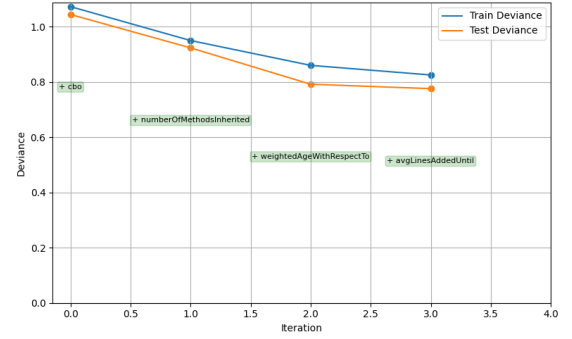
As the log-transformation might have destroyed the (linear) relationship of certain variables — especially if it is with the response variable — we will compare performance of the log-transformed and normalized data to the just normalized data.

Note that to ensure a valid log-transformation, we added a constant to each of the features, to ensure that each feature is greater than zero. This is part of our log-transform step.

Normalized Data



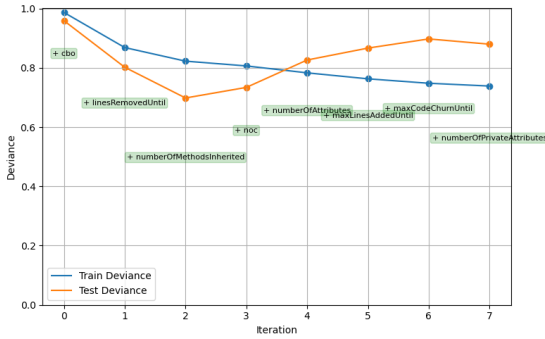
(a) Using AIC.



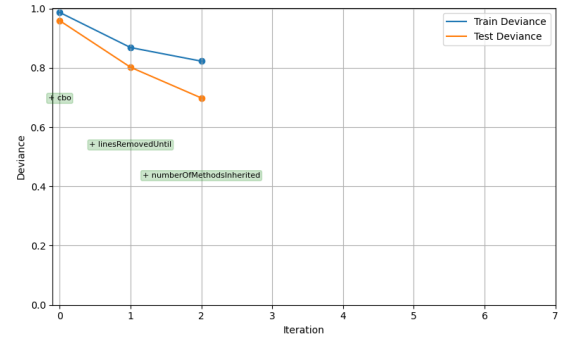
(b) Using BIC.

Figure 2.2: Train and test deviance of the Poisson regression when using stepwise model selection when using the normalized featured. Note that the green boxes starts with + denote which feature has been added to the model.

Log-transformed & Normalized Data



(a) Using AIC.



(b) Using BIC.

Figure 2.3: Train and test deviance of the Poisson regression when using stepwise model selection when using the log-transformed and normalized featured. Note that the green boxes starts with + denote which feature has been added to the model.

2.2 Poisson regression

2.2.1 Stepwise model selection

We use Poisson regression to model the number of bugs for each function. To find the best set of features to use for the Poisson regression, we performed model selection using both forward and backward stepwise selection, starting from a model with only an intercept.

To find the best model from the training features alone, we scored each found model using both AIC and BIC, as we are also interested in understanding when one is better than the other.

Normalized Data. Let us first look at the results from the Poisson regression models found using stepwise model selection. In Figure 2.2, we have plotted the train and test poisson-deviation at each step, while using

Search Score	Dataset	Train Deviation ($\downarrow$)	Test ($\downarrow$)
AIC	Normalized	0.81064	0.81010
	Log-Transformed & Normalized	0.73834	0.87946
BIC	Normalized	0.82569	0.77617
	Log-Transformed & Normalized	0.82231	0.69807

Table 2.1: The mean Poisson deviation values for the best **Poisson regression model** obtained using **Stepwise model search** for the different datasets. We highlight in bold the best value of the table for each of the data splits.

the AIC and BIC scores to estimate when to stop the search. We report on the final mean Poisson deviance in Table 2.1.

First, when using AIC, we find a model using 5 features, namely `cbo`, `numberOfMethodsInherited`, `weightedAgeWithRespectTo`, `avgLinesAddedUntil`, and `noc`. Interestingly, while the Poisson deviance decreases with each feature that is being added, we observe that adding the `noc` features makes the test deviance increase, while the train deviance decreases. This is a sign of overfitting.

Interestingly, when using BIC to score the models, we observe that the same features are being added in the same order. However, when adding a fifth feature, including `noc`, the BIC score increases, and therefore the search stops.

Log-Transformed and Normalized Data. We run the same code on the data we transformed to the log space. We plot the train and test mean Poisson deviance in Figure 2.3 and report on the final mean Poisson deviance in Table 2.1.

First, we remark that, compared to the normalized data, using AIC for scoring the search finds 7 features, whereas when using BIC, only 3 features are found.

Similarly as for the just normalized data, BIC stops the search as soon as the Poisson deviance of the test set increases, while finding the same sets of features.

Interestingly, we remark that first transforming the data to the log domain uncovers new relationships. This can be seen from the first model from both Figure 2.2 and Figure 2.3, which only consists of the `cbo` feature and an intercept. Whereas for the just normalized data, the train and test deviance is above 1, when using the log-transformed data, the mean Poisson deviance is below 1 for both the train and test set.

This clearly shows that transforming the features to the log space allows the linear relationship to explain more of the variance, and therefore have better predictions.

Analysis of the Parameters. Before looking at the Logistic regression, let us first investigate the values of the found parameters. From the lecture, we know that increasing the features of a sample $x \in \mathbb{R}^D$ by $\delta \in \mathbb{R}^D$ results in

$$\frac{\mathbb{E}[y \mid x + \delta]}{\mathbb{E}[y \mid x]} = \exp(\beta \cdot \delta). \quad (2.1)$$

We can easily understand the impact of each parameter on the prediction by constructing δ as follows: Let us investigate the increase of the raw feature by one unit. We can construct such a δ_i for each feature as

follows:

$$\delta_i = \frac{1}{\sigma_i}$$

as we have

$$\begin{aligned} \text{Normalized}(x) + \delta_i &= \frac{x - \mu}{\sigma} + (0, \dots, \underbrace{\frac{1}{\sigma_i}}_i, \dots, 0) \\ &= \frac{x - \mu}{\sigma} + \frac{1}{\sigma} (0, \dots, \underbrace{1}_i, \dots, 0) \\ &= \frac{x - \mu}{\sigma} + \frac{1}{\sigma} \delta \\ &= \frac{x - \mu + \delta}{\sigma} \\ &= \text{Normalize}(x + \delta) \end{aligned}$$

with δ denoting a one hot vector at the index of the feature of interest.

It is important to note that we are unable to perform such a simple analysis of the coefficient for the log-transformed data. This is because $\log(x + 1) \neq \log(x) + \log(1)$, and the increase of the a feature by one changes the prediction depending on x .

While we could compute the expected change in prediction by computing the probability of observing each value of a feature based on the data, we believe that the high-variance induced by the estimated probability would render any analysis meaningless.

Therefore, we only report on the influence of the coefficient for the normalized features.

Variable	Coefficient	$\frac{\mathbb{E}[y x+\delta_i]}{\mathbb{E}[y x]}$	p-value	95% CI
Intercept *	-0.800	-	< 0.001	[-1.017, -0.583]
cbo *	0.496	0.929	< 0.001	[0.419, 0.572]
numberOfMethodsInherited *	0.236	1.036	< 0.001	[0.148, 0.324]
weightedAgeWithRespectTo: *	0.392	1.007	< 0.001	[0.245, 0.538]
avgLinesAddedUntil: *	0.205	1.016	0.002	[0.075, 0.335]
noc	0.109	1.401	0.052	[-0.001, 0.219]

Table 2.2: Coefficients for the Poisson regression using **Stepwise model section**, when using the **normalized data** and **AIC**. It is the model found in the last step in Figure 2.2 (a). The features with * are also contained in the model found when using BIC.

Let us now look at the coefficients and the learned impact on the prediction. We report on the coefficients of the trained Poisson regression model, when using AIC to score the search in Table 2.2. Note that we do not report on the coefficients when using BIC as a search score due to their similarities. While the model found by using BIC has slightly different parameter values (e.g. the parameter for `cbo` is 0.507 in the BIC model, compared to 0.496 in the AIC mode, which results in a very similar performance), we hypothesize that the difference in the paramers is due to the number of variables added to model; As the AIC model has more variable, adding `noc` explains more of the variance in the training set, and therefore impacts the other parameters.

We report the coefficients of the Poisson regression model found by using the log-transformed data in Table 2.3. While these reported coefficient are from the model returned by using AIC for the search (see the model in Figure 2.3 (a)), the values for the simpler model found by using BIC have very similar coefficients as the one reported in Table 2.3. Therefore, we omit to report on these coefficients.

Variable	Coefficient	$\frac{\mathbb{E}[y x+\delta_i]}{\mathbb{E}[y x]}$	p-value	95% CI
Intercept *	-0.998	-	< 0.001	[-1.250, -0.746]
cbo *	0.616	-	< 0.001	[0.354, 0.878]
linesRemovedUntil: *	0.698	-	< 0.001	[0.365, 1.030]
numberOfMethodsInherited *	0.231	-	0.001	[0.094, 0.368]
noc	0.105	-	0.118	[-0.027, 0.236]
numberOfAttributes	0.434	-	0.003	[0.150, 0.718]
maxLinesAddedUntil:	-0.815	-	0.012	[-1.451, -0.179]
maxCodeChurnUntil:	0.469	-	0.067	[-0.033, 0.971]
numberOfPrivateAttributes	-0.162	-	0.143	[-0.379, 0.055]

Table 2.3: Coefficients for the Poisson regression using **Stepwise model section**, when using the **log-transformed and normalized data** and **AIC**. It is the model found in the last step in Figure 2.3 (a). The features with * are also contained in the model found when using BIC.

We point out that even though some coefficients in Table 2.3 are negative, increasing that feature by one unit might still increase the predicted value.

2.2.2 L_1 -Lasso Regularization

We now aim to compare the models found by using L_1 -lasso regularization to the stepwise model search. Normalizing the features to be centered around zero with a mean of 1 is especially critical here, as we aim for the L_1 -lasso regularization to be equal for all parameters;

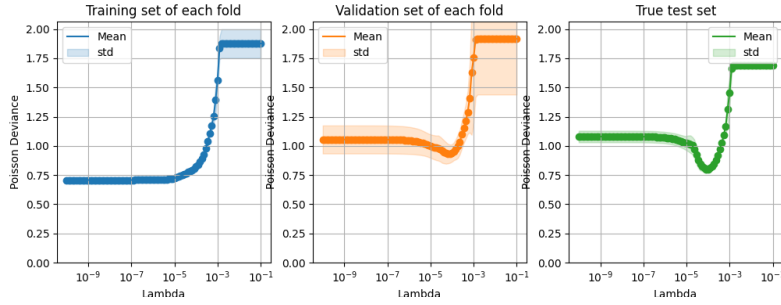
We use the same data pre-processing as in the section above and report on the mean and standard deviation of the mean Poisson deviation from 5-fold cross validation of the train set. We report on the deviation of the train set of each fold, the validation set from each fold (the excluded samples from the train set), and the deviation on the full test set in Figure 2.4.

Note that with the hyper-parameter we tested, no convergence of the training was observed. We believe that our learning rate $3 \cdot 10^{-3}$ is too high and might therefore jump over the local minimum, or we did not run the optimization for enough steps (20 000 steps).

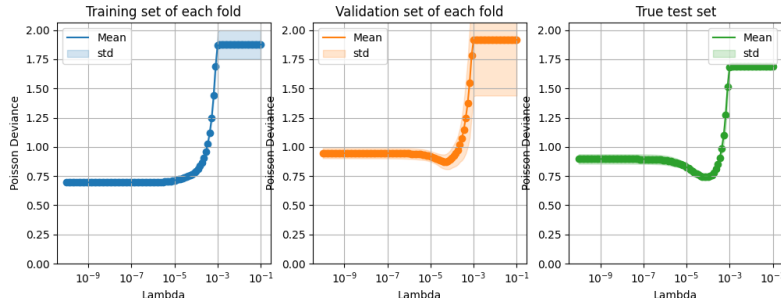
From Figure 2.4, we clearly see that lowest value for the mean Poisson deviance is convex at $\lambda \approx 10^{-4}$. In fact, we find that the best value for λ is $1 \cdot 10^{-4}$ for the just normalized data, and $8 \cdot 10^5$ for the log-transformed data. Using these values, we obtain the following values:

As we can use L_1 -regularization for feature selection, we plot the coefficients for the individual parameters over the values of λ . As we have 32 parameters in total, we add the plots in the Appendix (see Figure A.5 and Figure A.6), as, while they are interesting, these plots are not at the core of this question.

We remark that, especially in the case where we use the log-transformed data (see Figure A.6), all the features found by the stepwise search with the BIC score are have non-zero coefficients in the lasso regression. On



(a) Normalized Data.



(b) Log-Transformed and Normalized Data.

Figure 2.4: The mean and standard deviation of the mean Poisson deviation score for the subset of the training set of each fold (left, blue), the hold out set for of the train set for each fold (middle, orange), and the full test set (right, green).

	Train	Test
Normalized	0.82125	0.79864
Log-Transformed & Normalized	0.77761	0.74252

Table 2.4: The mean Poisson deviation values for the best **Poisson regression model** obtained using L_1 -lasso for the different datasets. We highlight in bold the best value of the table for each of the data splits.

the other hands, many values found by the AIC stepwise search have coefficients of zero.

However, lasso-regression, if interpreted as a feature selector, finds a different set of features as stepwise search with BIC. For example, `numberOfVersionsUntil`: has non-zero coefficients at at the optimal value for λ , which is not included in the set of features found by any search algorithm for stepwise model search.

2.2.3 Discussion

AIC vs BIC. It is clear that BIC outperforms AIC. Not only does BIC stops as soon as the test deviance increases by just looking at the training set, which is not the case for AIC.

Furthermore, while AIC keeps adding features to the model, we remark that every feature from the BIC model is included in the set of features found when using AIC (see Table 2.2 and Table 2.3).

Therefore, together with the results from Table 2.5, which state that AIC overfits to the training set, whereas

BIC is more robust to overfitting, we can conclude that, empirically, BIC is better at scoring models than AIC.

Coefficients of the Model. Interesting, when looking at the coefficients of the learned model, we see that a very delicate balance is being optimized. For example, when looking at the coefficient for the `cbo` variable, we see that increasing it has a negative impact on the prediction, even though it has a positive coefficient (see Table 2.6). On the other hand, the oracle model for the binary classification of whether a bug is included in the code has a value around 6 times higher than in the Poisson regression model found by stepwise model selection (see Table A.6 in the Appendix).

This does not mean that `cbo` cannot give information on the number of bugs included in the code, but as the other variables, such as `numberOfMethodsInherited` or `weightedAgeWithRespectTo:` might better modeling capabilities of the number of bugs, removing the information of `cbo` can the Poisson regression model to better model the number of bugs.

P-values. We report on the p-values of the coefficients. While we found that AIC performs worse than BIC, we further notice that, every feature added additionally by AIC has a high p-value.

For example, in Table 2.6, we have a p-value of 0.052 for the `noc` variable, which makes the test deviance increase. Therefore, not adding it is better.

Furthermore, using a threshold value of 0.05 or 0.01, we would reject that `noc` has an influence on the prediction of the number of bugs.

Comparison of L_1 -regularization and Stepwise Model Selection. We see that the model found by stepwise model selection with BIC as scoring outperforms the best model found by L_1 -regularization (compare Table 2.1 and Table 2.4).

Furthermore, we see that L_1 -regularization not only selects way more features in the model with the best value for λ , estimated on the hold out set, but out of the selected features, all of them overlap with the features found by stepwise search (see the coefficients in Figure A.5 and Figure A.6).

2.3 Logistic Regression

We constructed a binary target variable `bugbin`, where `bugbin` = 1 if `bugs` $\geq$ 1 and 0 otherwise. We perform the same analysis as done in the previous section, however, instead of reporting on the mean Poisson deviation, we report on the mean accuracy.

2.3.1 Stepwise Model Selection

We observe the same relationship between the AIC and BIC search as in Section 2.2.1. Therefore, we will only report on the stepwise search using BIC and omit to report on the performance of the AIC search. We plot in Table 2.5 the evolution of the accuracy of the different models using stepwise search. We omit a detailed discussion of the plots, as we would use the same arguments denoted in Section 2.2.1.

We therefore obtain the following accuracies for the stepwise search:

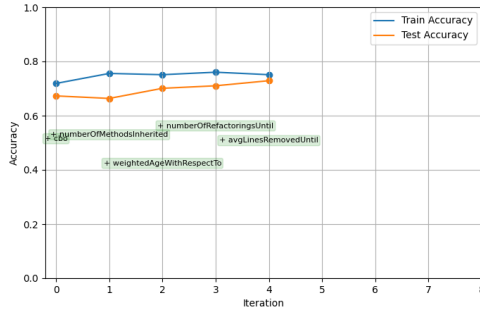
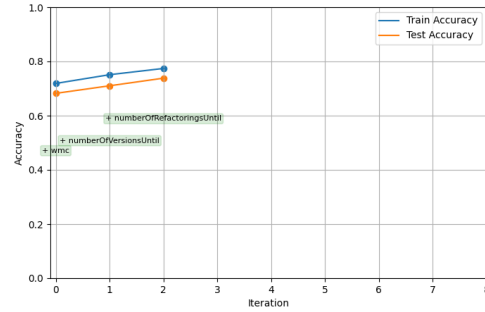

 (a) Using **Normalized Data**.

 (b) Using **Log-Transformed & Normalized**.

 Figure 2.5: Train and test deviance of the **Logistic regression** when using **stepwise model selection** with **BIC**. Note that the green boxes starts with + denote which feature has been added to the model.

Set	Train ($\uparrow$)	Test ($\uparrow$)
Normalized	0.751152	0.728972
Log-Transformed & Normalized	0.774194	0.738318

 Table 2.5: The mean accuracy for the best **Logistic regression** model obtained using **stepwise model search** for the different datasets. We highlight in bold the best value of the table for each of the data splits.

Similarly, we report on the values of the coefficients and their influence on the prediction in Table 2.6 for the normalized dataset and in Table 2.7 for the log-transformed and normalized features.

Variable	Coefficient	$\frac{\mathbb{E}[y x+\delta_i]}{\mathbb{E}[y x]}$	p-value	95% CI
Intercept	-0.406	—	0.024	$[-0.759, -0.054]$
cbo	1.289	0.963	< 0.001	$[0.777, 1.801]$
numberOfMethodsInherited	0.694	1.095	0.012	$[0.153, 1.235]$
weightedAgeWithRespectTo:	0.549	1.021	0.004	$[0.178, 0.921]$
numberOfRefactoringsUntil:	-0.558	3.951	0.008	$[-0.969, -0.147]$
avgLinesRemovedUntil:	0.728	0.959	0.019	$[0.119, 1.337]$

 Table 2.6: The coefficients of the Logistic regression model found using **stepwise search** with **BIC** for the **normalized** data.

2.3.2 L_1 -Lasso Regularization

We run the same code as in Section 2.2.2, namely logistic regression with 5-fold cross validation on the training set.

As shown in Figure 2.4, using both datasets results in almost equal plots of the the accuracy (deviation in the Figure) when plotted against the value of λ . Therefore, we move the accuracy-vs-lambda-plot for the normalized data to the Appendix (Figure A.8), and report on the log-transformed and normalized data in Figure 2.6.

Surprisingly, we observe that the estimated test accuracy from the fold does not (highly) correlate with

Variable	Coefficient	$\frac{\mathbb{E}[y x+\delta_i]}{\mathbb{E}[y x]}$	p-value	95% CI
Intercept	-0.621	—	0.001	$[-0.974, -0.268]$
wmc	0.986	—	< 0.001	$[0.506, 1.466]$
numberOfVersionsUntil:	1.014	—	< 0.001	$[0.512, 1.517]$
numberOfRefactoringsUntil:	-0.482	—	0.011	$[-0.853, -0.112]$

Table 2.7: The coefficients of the Logistic regression model found using **stepwise search** with **BIC** for the **log-transformed & normalized** data.

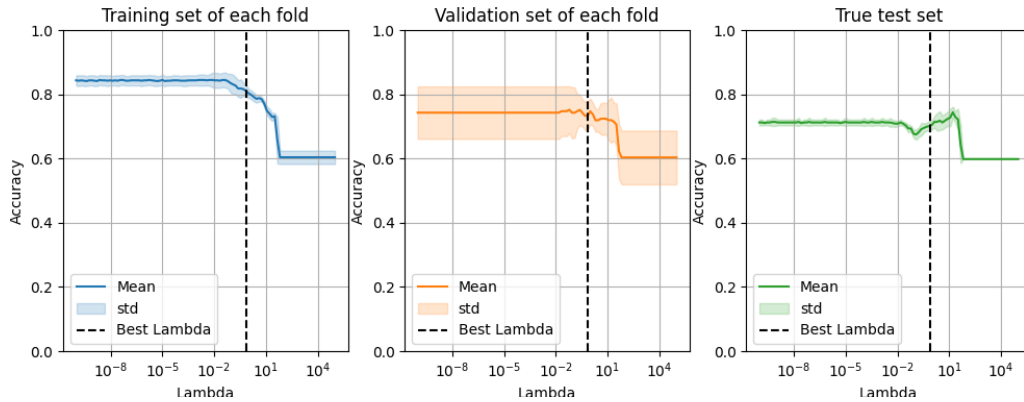


Figure 2.6: Accuracy w.r.t. λ for the L_1 -lasso **logistic** regression for the log-transformed and normalized data.

the true test set accuracy. Therefore, the value we find for best lambda, based on the estimated validation accuracy, is not at a maximum of the true test accuracy.

We again report on the optimized coefficients w.r.t. λ in Figure A.7 in the Appendix. We also add a table containing the coefficients of the logistic regression model learned with the best λ value for the normalized dataset in the Appendix (Table A.6).

Using the best value for λ that we found using the hold out set of each fold, we obtain the following accuracies:

Set	Train ($\uparrow$)	Test ($\uparrow$)
Normalized	0.81105	0.74766
Log-Transformed & Normalized	0.81567	0.719626

Table 2.8: The mean accuracy for the best **Logistic** regression model obtained using L_1 -**regularization** for the different datasets. We highlight in bold the best value of the table for each of the data splits.

2.3.3 Discussion

First, let us discuss the results from the L_1 -lasso regression.

Discrepancy in Table 2.8. One of the most surprising results is contained in Table 2.8, where the log-transformed data got outperformed by the just normalized data. This goes against the trend we

observed all along the report, namely that the log-transformation of the data achieves better results than the normalized results.

However, we can explain the results from Table 2.8. To this end, let us first look at the accuracy vs lambda plot of the logistic regression using the log-transformed data (Figure 2.5). In this Figure, we can clearly see that best accuracy on the test set is obtained with $\lambda \approx 16.0$, where the mean accuracy jumps to approximately 0.7439. However, the highest value on the estimated accuracy on the hold out set (middle plot of the Figure) is far from the optimal λ for achieving maximal test accuracy.

Therefore, while the logistic regression on the log-transformed and normalized dataset has a potential for a higher accuracy, the logistic regression on the normalized data reaches a better peak.

Found Features with L_1 -Regularization. Let us now look at the coefficients for the best logistic regression model found when adding L_1 -regularization on the normalized data (Table A.6).

Note that interpreting the value, or even the sign, of the coefficients of a logistic regression fitted on log-transformed data is not feasible, as the impact of the value change w.r.t. the value of the input.

To gain insights from this analysis, we abstain from investigating the coefficients of the model found at $\lambda = \text{Best Lambda}$ from Figure A.8, as we see from Figure A.5 and Figure A.6 that for extremely low λ values, the variance is very high.

Furthermore, if $\lambda \approx 0$, we may risk overfitting on the training data, and therefore obtain a meaningless model. Therefore, we cheat and use $\lambda = 1$, which maximizes the test accuracy for this case.

When looking at the three most important features, we have that a coefficient for `cbo`, which denotes the Coupling Between Objects, and has a positive coefficient. This makes sense, as increasing the number of objects coupled together, if one of these objects has a bug, then all of them have a bug. Similarly, `noc`, which stands for Number Of Children, has a strong influence on the prediction of a bug. This follows from the last argument.

Surprisingly, in this model, the number of methods (`numberOfMethods`) has a negative impact on the prediction.

We remark that many many features that we expect to correlate with the prediction have a strong (positive or negative) influence on the prediction. For example, `numberOfRefactoringsUntil`: reduces the odds of predicting a bug by 75% for every time a refactoring has been done. This is biased data, as code is being refactored if it contains bugs, or is prone to contain bugs.

We remark that we are unable to relate all features with respect to each other, as while the features are normalized, their value might not change the same way. For example, changing the number of lines of code will result in a much smaller difference in input (which we denote as Δx) compared to `numberOfRefactoringsUntil`: for example.

A joint analysis is way beyond the scope of this assignment, as we would need to look at the distribution of the features and the coefficients jointly, which, for 32 feature, is very vast.

Comparison between L_1 -Regularization and Stepwise Model Selection. Let us now compare the results of the L_1 -regularization and the results from the stepwise model selection for the logistic regression.

Comparing the best found accuracies, we see that, while the log-transformed dataset outperforms the just normalized dataset when using stepwise model selection (see Table 2.5), the trend is reversed¹ in the L_1 -regularization case (see Table 2.8).

Furthermore, we see that L_1 -regularization allows to find better performing models than stepwise model selection. We will discuss this in more detail in Section 2.4.

The most obvious difference between both search methods is the number of features found. Whereas stepwise feature selection finds very few features (when using BIC as scoring), L_1 regression sets coefficients to zero only for high values of λ , where the model then degrades due to the added constraints by the regularization.

When looking at the coefficients, interestingly, the fitted models found by the stepwise model selection have drastically different coefficients for some of the parameters. For example, on the normalized data, stepwise selection finds a coefficient of 1.289 for the `cbo` variable (see Table 2.6), in the *oracle* model, we have a coefficient of 3.020, which drastically changes the influence of the feature on the prediction.

Investigating whether stepwise model selection could benefit from having one of the coefficient fixed from the oracle model would be an interesting future work.

Accuracy Metric. While the accuracy metric is a good metric to observe how good a classification model performs, we wish to comment on the metric used.

Other popular metrics used in classification are the F_1 score, the selectivity, and specificity. Which one is optimal depends on the application of the model.

We believed that, for this task, the accuracy gives us enough information to compare the different model. However, if someone wants to, for example, develop a software that raises a warning if the probability for a bug in the function is high, then false positives are really undesired in the general case. Note that some cases have the type of error priority flipped; For example, when programming for NASA, raising a false positive is less impactful than having a false negative, and shipping code that contains a bug.

2.4 Discussion — Differences Between L_1 -regularization and Stepwise Model Selection

Let us now comment on the differences between the two search algorithms. A question one might — rightfully — ask is *which search algorithm is better?* We believe that this question has no simple answer.

In this report alone, we have seen two cases of when one search performs better than the other; In our case, when modeling the number of bugs in the functions, we find that stepwise model selection outperforms L_1 -regularization, whereas for predicting the existence of at least one bug, we see have found that L_1 -regularization outperforms stepwise model selection, with even more stable accuracies.

We hypothesize that, in the case of the modeling of the number of bugs, simpler models outperform complex ones. We argue that this is due to the simpler model capturing the general trend, instead of potential noise that might correlate with the response variable in the train set but not in the test set.

¹See previous argument.

If a model is dependent on a lot of variables, the probability that one of the variables changes by chance (e.g., because of noise, or hidden confounders), is very high, and therefore, the prediction might be off because of this arbitrary change. Note that this is different from the overfitting phenomenon, which also contributes to a model having a higher number of features. However, a model having a lot of features does not necessarily overfit, whereas the probability that a model has a lot of features, given that it has overfit, is much higher.

On the other hand, for the binary prediction case, we can make use of more features, as the magnitude of the prediction is not very relevant, but rather if the prediction crosses the 0.5 boundary. Therefore, if some noise is being added to the features, a complex model could still have the same prediction in the logistic regression case, whereas the difference to the original prediction could be quite large for the Poisson regression.

Appendix A

Appendix

A.1 Statement on the Useage of Gen. AI

- **Felix Céard-Falkenberg**

Unless stated otherwise, all code, text, and math derivations have been entirely thought and written by me with no external help — both for gen. AI and wolframalpha anc co.

We used Claude 4.5 Opus for generating the plots for Figure 1.2 as the plot is not a core of the assignment, and only aids in understanding our own results. Note that we (heavily) modified the code generated by Claude.

For question 2 of the assignment, we used ChatGPT 5.2 to help us with some of the math of the derivations. The end product is 90% my own, indepent work. Most of the A.I. being used for question 2 got scrapped as it did not add any value to the report. The most notable scrapped work was finding some result to $\mathbb{E} \left[\frac{f_1^2}{2f_2} \right]$, but as both are dependent, only very limited results were found by ChatGPT.¹

We used Claude 4.5 Opus for generating the plots for Figure 1.2 as the plot is not a core of the assignment, and only aids in understanding our own results. Note that we (heavily) modified the code generated by Claude.

For Part B, we used ChatGPT to insert some value from lists into latex tables, but we made sure that all values lined up.

- **Yiquan Hu**

For Part B, we used ChatGPT to generate required code for stepwise selection with the permissions stated in the assignment instructions and also consulted ChatGPT for general suggestions on parameter adjustment. All analysis, results and interpretations were produced by us. We only used ChatGPT lightly refine the academic word of some sentences without adding new content.

¹This may also be due to my bad prompting.

A.2 Additional Tables

$$p_i \sim 0.02 \cdot \text{Beta}(1, 3)$$

		$N = 500$	$N = 1000$	$N = 5000$
$\hat{N}$		572.21±249	1102.16±327	5165.82±579
Error (%)		14.44	10.22	3.32
$\hat{\sigma}_{\hat{N}}$		223.18±168	288.20±146	568.22±96
C.I.	$\alpha = 0.05$	[296.06±80; 1241.40±822]	[688.71±147; 1861.17±749]	[4189.85±421; 6432.61±798]
	$\alpha = 0.01$	[248.36±58; 1610.48±1203]	[602.47±115; 2212.82±975]	[3932.12±381; 6903.90±882]
Coverage (%)	$\alpha = 0.05$	94.60±22	94.00±23	94.60±22
	$\alpha = 0.01$	98.90±10	98.80±10	98.90±10
Bias ($\approx$)		5 214	10 436	27 495
Variance ($\approx$)		78 120	104 540	332 210
MSE ($\approx$)		83 334	114 977	359 705

Table A.1: Result from 1000 MC simulations for estimating $\hat{N}$. For each value, we report on the mean (black) and standard deviation (gray). We omit the digits after the decimal point for the standard deviation for brevity. The Error (%) is the absolute difference between the estimated and the true value of N , divided by the true value of N .

$$p_i \sim 0.01 \cdot \text{Beta}(2, 2)$$

		$N = 500$	$N = 1000$	$N = 5000$
$\hat{N}$		582.90±314	1093.58±294	5128.88±565
Error (%)		16.58	9.36	2.58
$\hat{\sigma}_{\hat{N}}$		235.15±275	281.72±121	561.35±93
C.I.	$\alpha = 0.05$	[297.63±84; 1303.01±1391]	[687.38±139; 1831.89±634]	[4164.29±411; 6379.84±776]
	$\alpha = 0.01$	[249.11±59; 1716.36±2272]	[602.16±110; 2171.47±808]	[3909.42±373; 6844.99±858]
Coverage (%)	$\alpha = 0.05$	95.30±21	95.50±20	95.40±20
	$\alpha = 0.01$	99.10±9	99.10±9	98.80±10
Bias ($\approx$)		6 871	8 757	16 608
Variance ($\approx$)		131 042	94 106	323 905
MSE ($\approx$)		137 913	102 864	340 514

Table A.2: Result from 1000 MC simulations for estimating $\hat{N}$. For each value, we report on the mean (black) and standard deviation (gray). We omit the digits after the decimal point for the standard deviation for brevity. The Error (%) is the absolute difference between the estimated and the true value of N , divided by the true value of N .

$$p_i \sim U(0, 0.01)$$

		$N = 500$	$N = 1000$	$N = 5000$
$\hat{N}$		517.55±100	1022.24±129	5057.81±290
Error (%)		3.51	2.22	1.16
$\hat{\sigma}_{\hat{N}}$		95.08±31	130.52±27	285.44±26
C.I.	$\alpha = 0.05$	[374.85±57; 757.81±181]	[809.99±88; 1328.62±193]	[4542.75±244; 5664.70±345]
	$\alpha = 0.01$	[343.31±49; 863.09±218]	[757.85±78; 1450.54±219]	[4397.59±232; 5876.88±364]
Coverage (%)	$\alpha = 0.05$	95.00±21	95.20±21	94.60±22
	$\alpha = 0.01$	98.80±10	99.60±6	98.40±12
Bias ($\approx$)		307	494	3 342
Variance ($\approx$)		10 013	17 769	82 162
MSE ($\approx$)		10 321	18 263	85 504

Table A.3: Same configuration as for Table 1.1, but we run the simulation for twice as long ($T = 80$).

A.2.1 Proof of the dependency of f_1 and f_2

Custom Note:

The idea of this proof is to be credited to ChatGPT.

The current proof is just our own interpretation of the proof generated by ChatGPT, which we have deleted in the mean time.

We can model the probability distribution of all f_k jointly as a multinomial distribution.

In statistics, we can define two variables as independent if their covariance is zero.

From the multinomial distribution, we know that the covariance between f_i and f_j is given by

$$\text{Cov}(f_i, f_j) = -Np_i p_j .$$

Therefore, we have that

$$\begin{aligned} \text{Cov}(f_1, f_2) &= -Np_1 p_2 \\ &= -N \cdot \Pr(X_i = 1 \mid T = t) \cdot \Pr(X_i = 2 \mid T = t) \\ &= -N \cdot \text{Binomial}(T, 1, p) \cdot \text{Binomial}(T, 2, p) \end{aligned}$$

Naturally, we see that the Covariance is not zero, but slightly negative, and therefore the two variables are not independent.

For our four probability distributions, we have the following covariance values:

From the multinomial distribution, we know that the variance of f_i is given by

$$V(f_i) = Np_i(1 - p_i) . \tag{A.1}$$

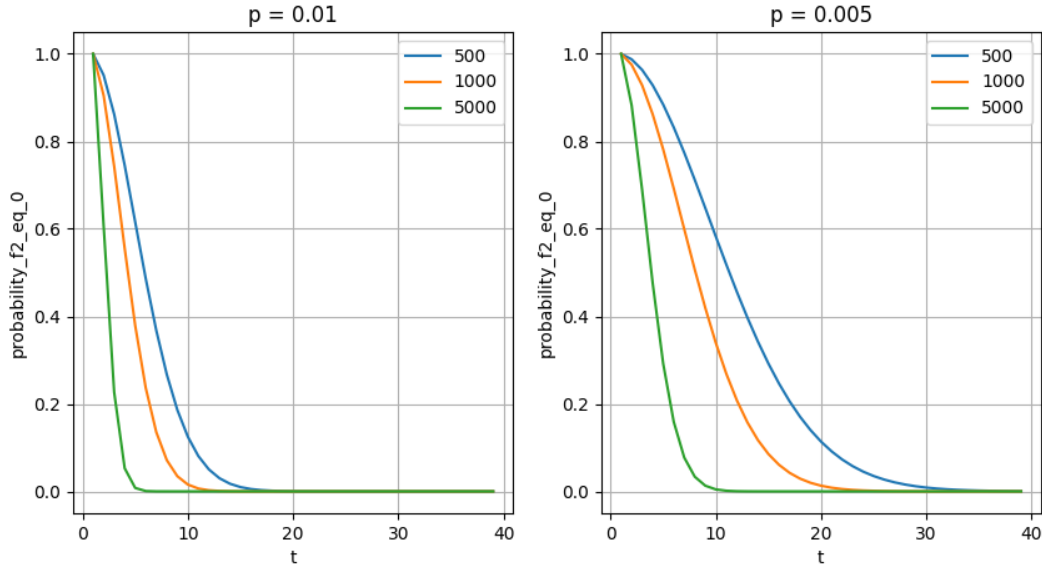


Figure A.1: Probability for $f_2 = 0$ over 40 times steps for different population sizes. Note that

	Cov(f_1, f_2)		
N	500	1000	5000
$p = 0.01$	-7.20	-14.39	-71.95
$p = 0.005$	-1.33	-2.65	-13.26

Table A.4: Covariance for f_1 and f_2 for $T = 40$ and $N \in \{500, 1000, 5000\}$ for different values of p .

Therefore, we can use Equation A.1 to compute the correlation between f_1 and f_2 and report it in Table A.5.

	Corr(f_1, f_2)		
N	500	1000	5000
$p = 0.01$	-0.1443	-0.1443	-0.1443
$p = 0.005$	-0.0568	-0.0568	-0.0568

Table A.5: Covariance for f_1 and f_2 for $T = 40$ and $N \in \{500, 1000, 5000\}$ for different values of p .

□

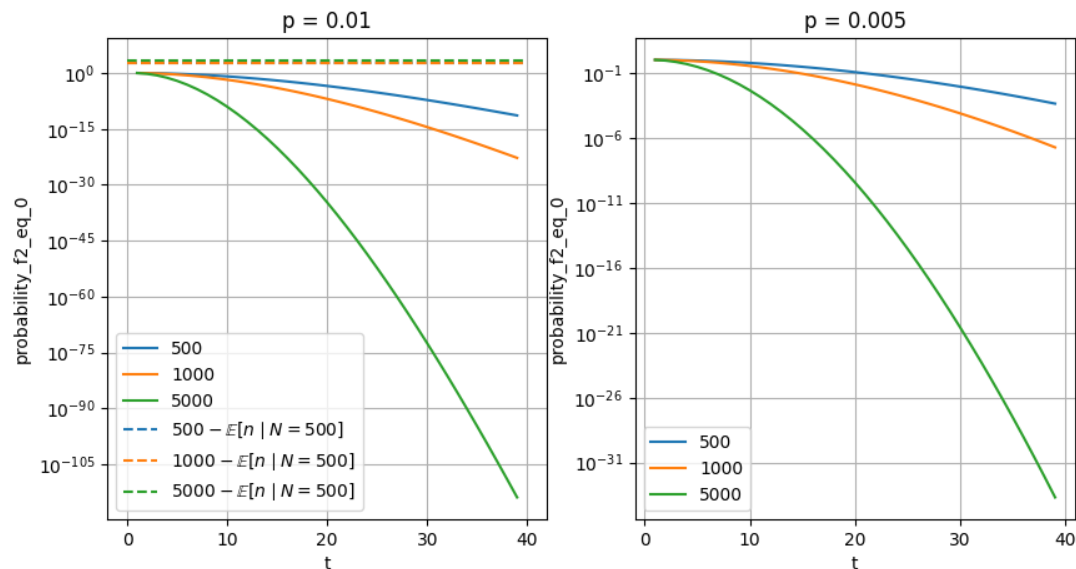


Figure A.2: The expected value for $\hat{f}_0$ conditioned on $f_2 = 0$ over 40 times steps for different population sizes. We multiply the expected value by the probability of $f_2 = 0$ at each time step.

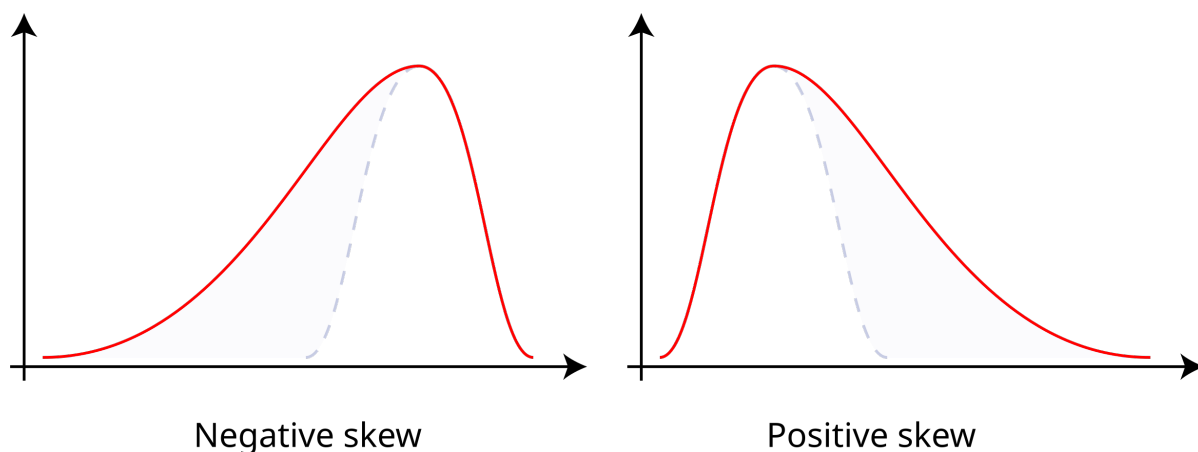
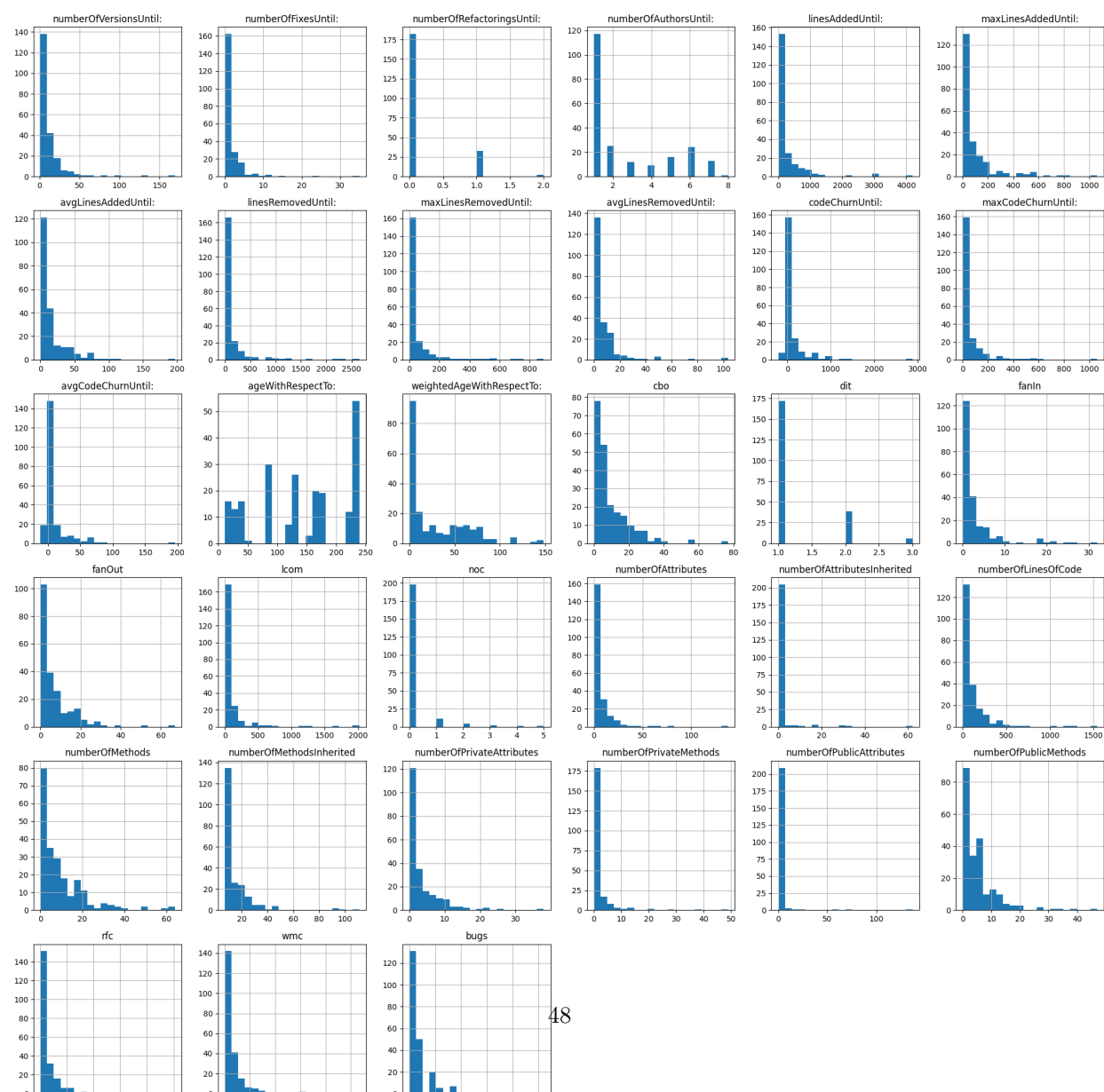


Figure A.3: Interpretation of the skewness of a distribution.

A.3 Additional Plots and Tables



Feature	Coefficient	$\frac{\mathbb{E}[y x+\delta_i]}{\mathbb{E}[y x]}$
noc	0.734	3.335
dit	0.567	3.219
cbo	3.020	1.319
numberOfPublicMethods	1.631	1.277
numberOfAuthorsUntil:	0.507	1.269
numberOfPrivateMethods	0.654	1.129
numberOfAttributes	0.932	1.071
numberOfAttributesInherited	0.335	1.059
wmc	2.731	1.050
avgLinesRemovedUntil:	0.567	1.044
numberOfPrivateAttributes	0.197	1.039
numberOfMethodsInherited	0.333	1.024
weightedAgeWithRespectTo:	0.384	1.012
linesRemovedUntil:	3.676	1.010
numberOfVersionsUntil:	0.175	1.009
maxLinesAddedUntil:	1.290	1.008
numberOfLinesOfCode	1.506	1.007
lcom	0.988	1.004
linesAddedUntil:	1.492	1.003
maxCodeChurnUntil:	-0.261	0.998
ageWithRespectTo:	-0.176	0.998
codeChurnUntil:	-0.819	0.997
avgLinesAddedUntil:	-0.197	0.992
avgCodeChurnUntil:	-0.252	0.988
maxLinesRemovedUntil:	-2.965	0.977
rfc	-3.512	0.966
numberOfPublicAttributes	-1.004	0.916
fanOut	-1.009	0.894
fanIn	-1.324	0.763
numberOfMethods	-3.550	0.727
numberOfFixesUntil:	-1.390	0.670
numberOfRefactoringsUntil:	-0.542	0.258

Table A.6: Coefficients of the best **Logistic** regression model found with L_1 -regularization for the **just normalized** dataset.

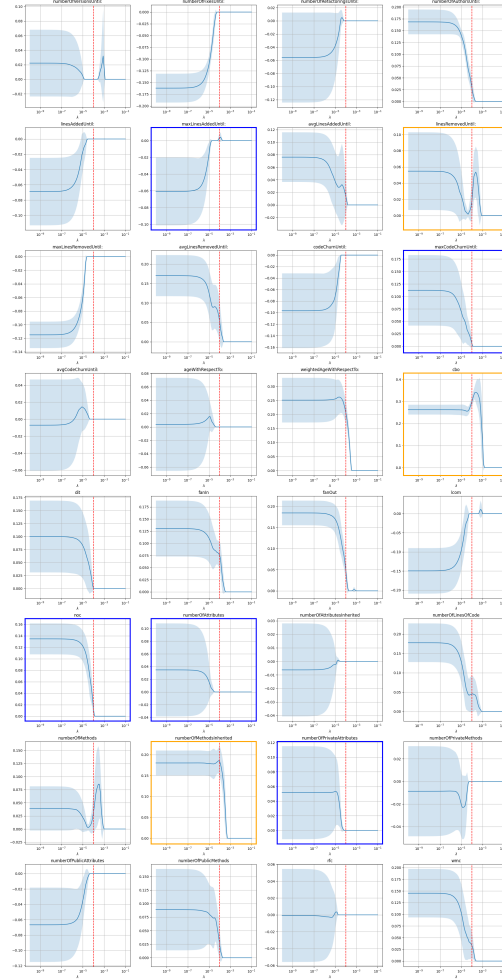
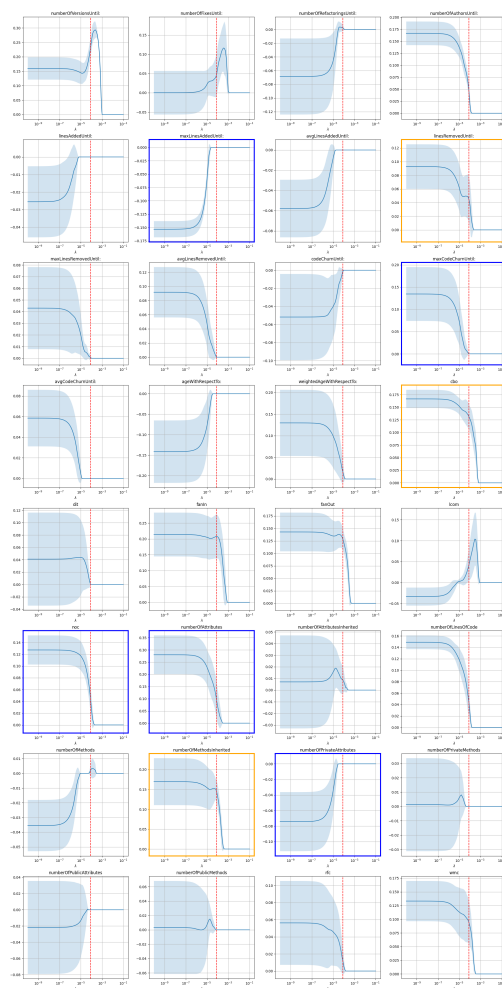


Figure A.5: The coefficients of the **Poisson** regression model for all 32 features w.r.t. λ when using the **normalized** data. The plots highlighted in **blue** are features found by the stepwise search model using AIC. We further highlighted in **orange** the features found by the stepwise search when using BIC. We advise the reader to view this report digitally to make the most of this plot.



51

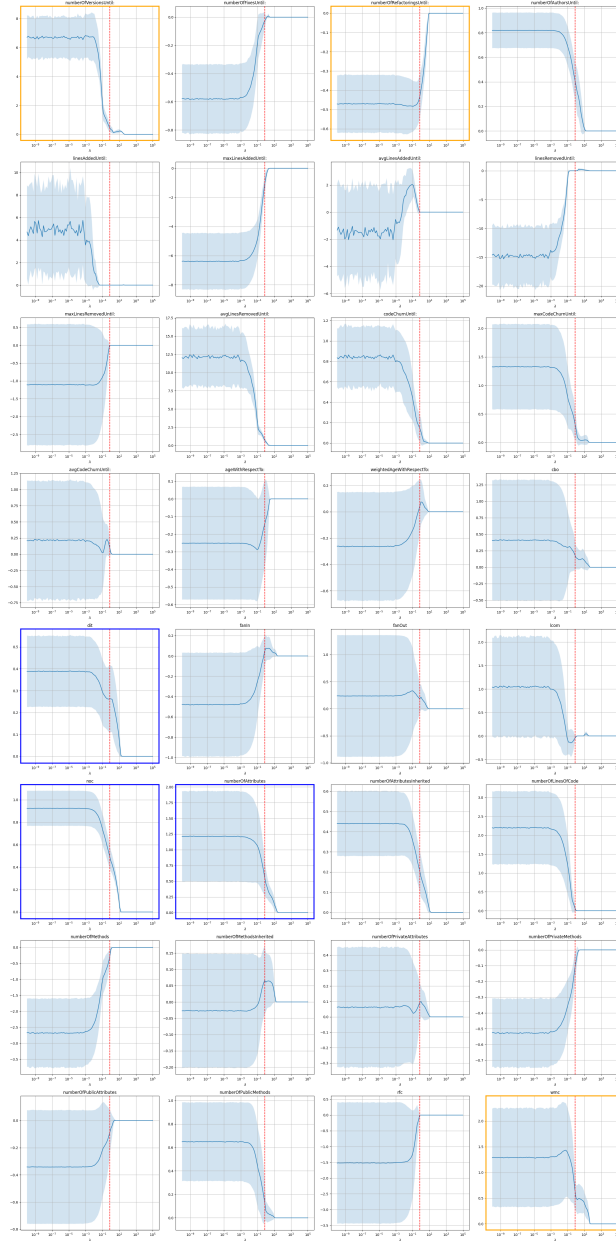


Figure A.7: The coefficients of the **Logistic** regression model for all 32 features w.r.t. λ when using the **log-transformed and normalized** data. The plots highlighted in **blue** are features found by the stepwise search model using AIC. We further highlighted in **orange** the features found by the stepwise search when using BIC. We advise the reader to view this report digitally to make the most of this plot.

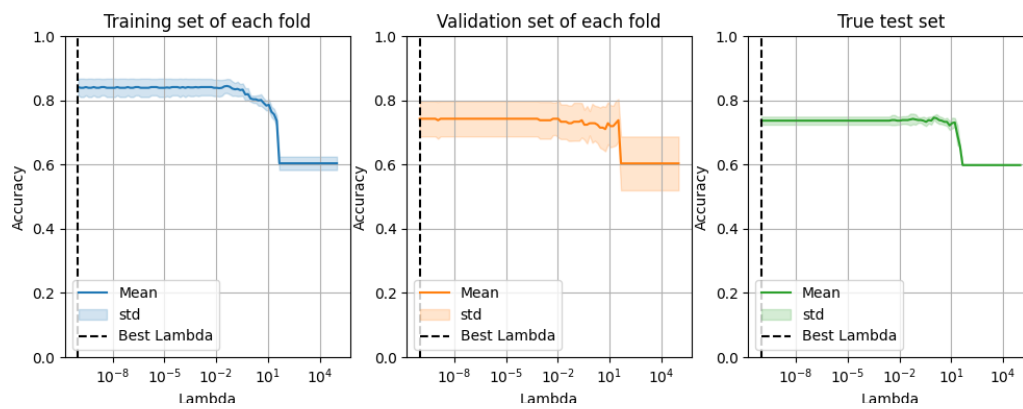


Figure A.8: Accuracy w.r.t. λ for the L_1 -lasso **logistic** regression for the just normalized data.